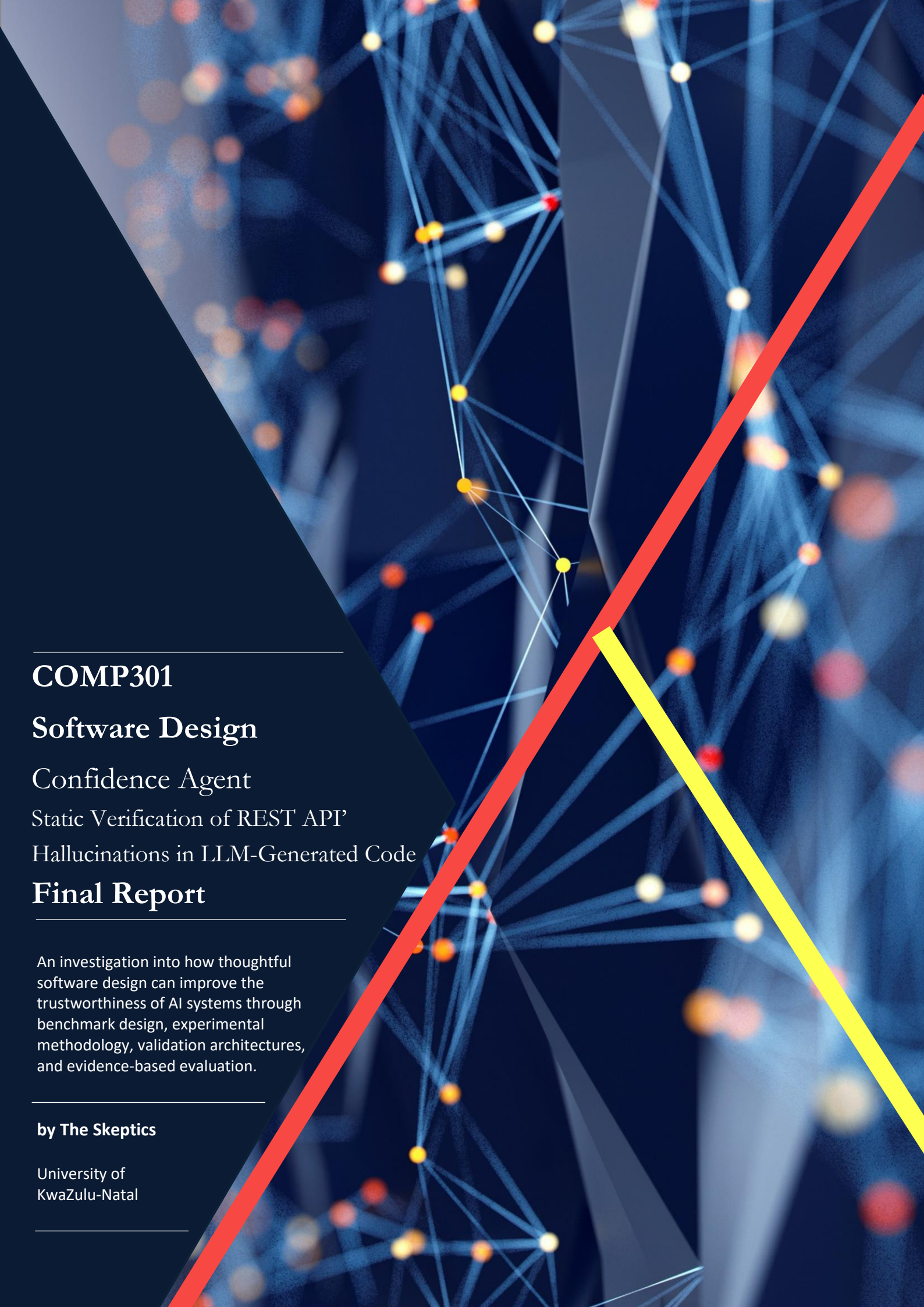


COMP301

Software Design

Confidence Agent

Static Verification of REST API' Hallucinations in LLM-Generated Code

Final Report

An investigation into how thoughtful software design can improve the trustworthiness of AI systems through benchmark design, experimental methodology, validation architectures, and evidence-based evaluation.

by The Skeptics

University of KwaZulu-Natal

Project Formalities

Domain: Static Verification of API Hallucinations in LLM-Generated Code

Group Name: The Skeptics

Group Members:

Member	Student number
Jawad Kachbal	223037045
Joshua Blom	224039813
Mahomed Valli Mahomed	224005878
Andiswa Hlengwa	221002430
Abiel Govender	224058851

Abstract

Large language models generate plausible code that calls web APIs which do not exist, at versions that have been retired, with parameters the provider never documented. This study is an empirical software engineering investigation into whether an agent-based validation architecture, a **Confident Agent**, can prevent such REST API hallucinations from reaching the user. The Confident Agent pairs a local model (Qwen3.5-9B) with a validation tool that checks every API claim in a candidate program against **frozen, hashed OpenAPI specifications** across six layers, summarises the evidence in an **Evidence-Weighted Confidence Score (EWCS)**, and applies a decision policy that delivers, flags, corrects once, or withholds the code.

A benchmark of 1,000 Python tasks over six public REST APIs was constructed with an executable answer key. Each of the 990 scored tasks was answered by the unmodified model and by the Confident Agent under three quantizations of the same model, giving 2,970 task pairs that repeat the same tasks rather than three independent replications. An evaluator that shares no code with the agent executed every candidate in a sandbox; human audits of 100 answer-key tasks and of 102 of the evaluator's decisions found no answer-key errors and no adjudicated disagreement on the primary outcome. In each run, the proportion of tasks in which hallucinated code reached the user fell from **50.8–55.2% to 1.6–2.3%** (exact McNemar $p < 10^{-129}$, Holm-adjusted), and the proportion receiving correct code rose from **24.9–36.9% to 50.4–51.5%**. The effect belongs to the combined architecture of prompt, verification, correction and abstention: the agent's first candidates, written before any verification feedback, still hallucinated in 46.7–48.9% of tasks.

Release decisions are governed by hard verification constraints, not by the confidence score: re-scoring every decision under five pre-specified weighting schemes changes only whether delivered code carries a flag, never what is withheld. The protection has costs. The agent abstains on 38–40% of tasks, including almost every task for the most recently changed API, and takes about **six times the time and energy** of the baseline. Verification establishes conformance to the API contract, not that the code does what the user asked: about one task in ten received code that met every check.

Keywords: LLM hallucination, REST API, OpenAPI, static verification, confidence estimation, abstention, McNemar test, energy cost.

Table of Contents

1 Introduction 1
1.1 Background and motivation 1
1.2 Problem statement 1
1.3 Aim and objectives 1
1.4 Contributions 2
1.5 Structure of the report 2
2 Research Question and Hypotheses 2
2.1 Research question 2
2.2 Hypotheses 2
2.3 Outcomes and guiding principles 3
3 Application Domain 3
3.1 Software APIs, REST and OpenAPI 3
3.2 Hallucination in generated code 4
3.3 Why external evidence is needed, and scope 4
3.4 Key terms in this study 5
4 Benchmark Design 5
4.1 Requirements 5
4.2 APIs and ground-truth evidence 5
4.3 Tasks, prompts and the answer key 6
4.4 What the benchmark measures 6
5 Baseline Evaluation 7
5.1 Baseline configuration 7
5.2 Baseline results 7
5.3 Why the baseline hallucinates 7
6 Confident Agent Design 8
6.1 Design principles 8
6.2 Architecture 8
6.3 Verification layers and evidence 9
6.4 The confidence metric 10
6.5 The decision policy 10
6.6 Validation of the apparatus 11
7 Experimental Methodology 12

7.1 Experimental design 12
7.2 Procedure 12
7.3 Measurements 13
7.4 Independent evaluation 13
7.5 Manual review of undetermined candidates 13
7.6 Human audits of the answer key and the evaluator 14
7.7 Pre-registration and statistical methods 14
8 Experimental Results 15
8.1 Headline results 15
8.2 Incorrect acceptance and incorrect rejection 16
8.3 Where in the architecture the reduction happens 16
8.4 Results by API, and where the intervention does not succeed 17
8.5 The confidence metric and the decision policy 17
8.6 Correction and agent behaviour 18
8.7 Cost of the intervention 19
9 Statistical Analysis 19
9.1 Primary hypothesis 19
9.2 Secondary hypotheses 20
9.3 Interpreting the significance tests 20
9.4 Robustness checks 20
10 Threats to Validity 20
10.1 Benchmark bias 21
10.2 Evaluator bias 21
10.3 Overfitting 21
10.4 Model drift 22
10.5 Confidence calibration 22
10.6 Internal validity of the comparison 22
10.7 Generalisability 23
10.8 Practical engineering trade-offs 23
11 Conclusion 23
11.1 Answer to the research question 23
11.2 Principal findings 23
11.3 Limitations and future work 24
11.4 Closing remarks 24

AI Disclosure 25
References 26
Appendix A: Summary of Results 28
Appendix B: Benchmark Details 29
B.1 Evidence and its provenance 29
B.2 Task construction and composition 29
Appendix C: Experimental Setup 31
C.1 Model and run configuration 31
C.2 Procedure details 31
C.3 Measurements 31
C.4 The independent evaluator 32
C.5 Manual review procedure 32
C.6 Changes to the evaluation plan 33
C.7 Statistical methods 33
C.8 Summary of threats to validity 34
Appendix D: Confident Agent Implementation 35
D.1 The Trust-by-Design patterns 35
D.2 Where the tool attaches, and why 35
D.3 Claim extraction 36
D.4 The six layers in detail 36
D.5 Frozen evidence, selected by host 37
D.6 The EWCS: signed score, weights, tiers and ceilings 37
D.7 Static and dynamic structure 39
Appendix E: Supplementary Results 41
E.1 Scorecard 41
E.2 Results by subgroup 42
E.3 Kinds of hallucination 42
E.4 The verifier as a hallucination detector 43
E.5 Discrimination and calibration of the EWCS 44
E.6 Sensitivity to the weights, thresholds and gate 44
E.7 Correction and agent behaviour 45
E.8 Behaviour beyond API correctness 46
E.9 The cost of the intervention 46
E.10 Quantization, run-to-run reliability and agreement between runs 49

E.11 Secondary outcomes and effect sizes 50
E.12 Cost statistics and reviewer agreement 51
Appendix F: Example Task and Prompts 52
Appendix G: Run Artefacts and Provenance 53
Appendix H: Error Kinds Used by the Evaluator 54
Appendix I: Manual Review Rules 55
Appendix J: Reproducing the Results 56
Appendix K: Glossary 57

List of Figures

Figure 1**:** Which API host and version each baseline answer called (three runs pooled). The model reproduces versions from its training data and extrapolates versions it has never seen. 8
Figure 2**:** Deployment and component view (UML). All components run on one workstation; the evaluator is independent of the toolbox. 9
Figure 3**:** Activity diagram (UML) of the decision policy. Hard rules precede the score; the report renderer releases code only when its hash matches the verified candidate. 11
Figure 4**:** Experimental pipeline. The answer key reaches only the evaluator; the model and the agent see the prompt alone. 12
Figure 5**:** The independent evaluator: sandboxed execution, three response scenarios and comparison with the answer key. 13
Figure 6**:** Each task's outcome under the baseline (left) and under the Confident Agent (right), three runs pooled; ribbon width is the number of task-runs. 15
Figure 7**:** Summary of results: supplementary charts (a)–(l). 28
Figure 8**:** How the six layers read one candidate: each claim is checked by one layer against one kind of evidence. 36
Figure 9**:** Class diagram (UML) of the verification core in toolbox 3.0.0. 39
Figure 10**:** Sequence diagram (UML) of one intervention task, including the single permitted correction and the reply request at the turn limit. 40
Figure 11**:** Scorecard: baseline versus Confident Agent for each quantization (990 tasks each). Electricity priced at R4.1109/kWh including VAT. 41
Figure 12**:** Rates per subgroup and run: hallucinated code reaching the user, correct code reaching the user, and code withheld by the verifier (B = baseline, CA = Confident Agent). 42
Figure 13**:** Kinds of API hallucination in the code reaching the user (three runs pooled; a task can have several kinds). 42
Figure 14**:** The verifier as a detector of hallucinated final candidates: quality measures, confusion matrices, and the processing time spent on its errors. 43
Figure 15**:** EWCS of the final candidate: ROC curve for correct versus class A, and score distributions with the decision thresholds (three runs pooled). 44
Figure 16**:** Decisions on the final candidate under the five planned schemes and with the uncertainty gate removed (three runs pooled). 45
Figure 17**:** Cost and outcome by the number of correction rounds the agent made (three runs pooled). 45
Figure 18**:** Deviations from the Confident Agent prompt recorded by the benchmark, per run. 46
Figure 19**:** Time, electricity (Rand per 1,000 prompts) and tokens per prompt, with 95% bootstrap intervals 47
Figure 20**:** What one correct answer, one prevented hallucination and one extra correct answer cost in electricity and time. 48
Figure 21**:** Where the Confident Agent's time and electricity go, by outcome. 49
Figure 22**:** Paired differences for the primary and secondary outcomes, with 95% bootstrap intervals over tasks (thick) and operation families (thin). 51

List of Tables

Table 1: Classes used to score the code that reaches the user. 4
Table 2: The terms defined by the project description and how this study realises them. 5
Table 3: APIs in the benchmark and the provenance of their evidence (retrieved 21 September 2026). 5
Table 4: Baseline: class of the code delivered to the user (990 tasks per run, pilot excluded). 7
Table 5: Checks of the verifier on known-valid and known-invalid code. 11
Table 6: Evaluator audit: agreement on 102 sampled decisions from the Q6_K run (Cohen's κ in brackets). 14
Table 7: Headline outcomes per run (990 tasks each; baseline → agent). 15
Table 8: Class A code at three stages: the baseline answer, the agent's first candidate (before verification feedback) and the code the agent delivered. 16
Table 9: Sensitivity analysis: outcomes on the final verified candidate (three runs pooled, 2,970 task-runs). 18
Table 10: Cost of the intervention (990 tasks per run; baseline → agent; electricity at R4.1109/kWh including VAT). 19
Table 11: Primary test: class A code reaching the user, baseline versus Confident Agent (990 paired tasks per run). 19
Table 12: Composition of the 990 scored tasks (the 10 pilot tasks are excluded from the analysis). 29
Table 13: The pinned model and run configuration (identical for baseline and intervention; read from the run manifests). 31
Table 14: What was measured for every task and condition. 31
Table 15: Changes to the evaluation plan after it was fixed on 27 September 2026. 33
Table 16: Statistical methods (evaluation plan §6). 33
Table 17: Summary of the main threats to validity and their mitigation. 34
Table 18: How the Confident Agent realises the Trust-by-Design patterns (Vorster, 2026a). 35
Table 19: Options for attaching verification to the model. 35
Table 20: Severity ranking of the layers and the rank-sum weights (total 15). 37
Table 21: Evidence tiers and factors. 38
Table 22: Maximum attainable EWCS when every applicable layer verifies (L1 tier 2, L2 tier 4). 38
Table 23: Share of decided final candidates (correct, A or B) that were correct, by EWCS band (three runs pooled). 44
Table 24: Class of the first verified candidate → class of the final candidate, for tasks in which the code changed. 45
Table 25: Task-behaviour checks on code that reached the user (share of programs that make a request). 46
Table 26: Cost of the intervention in time, energy and Rand (990 tasks per run; electricity at R4.1109/kWh incl. VAT). 47
Table 27: Repeat runs with identical settings: earlier series (superseded) versus final series. 49
Table 28: Agreement between the three final runs, task by task (990 tasks; Cohen's κ in brackets). 50
Table 29: Secondary outcomes (exact McNemar tests; intervals over tasks). 50
Table 30: Identifiers of the frozen inputs (SHA-256 prefixes; full hashes in the run manifests and the dataset). 53

Table 31**:** Kinds of error recorded in the code reaching the user, read from the evaluator's findings with fixed rules.....54
Table 32**:** Glossary.....57

1 Introduction

1.1 Background and motivation

Large language models (LLMs) are now part of everyday software development, and developers routinely ask them for code that integrates with third-party web services. Such code is a natural place for hallucination: output that appears plausible while being incorrect, misleading or fabricated (University of KwaZulu-Natal, 2026). A generated program may call a renamed endpoint, a retired host or a parameter the provider never documented, and it may run without an exception until it reaches production. Because the answer looks like correct code and is presented without hedging, the developer has little reason to doubt it. Trustworthiness is therefore becoming a software quality attribute of AI systems in its own right.

This study does not attempt to make a model stop hallucinating. It asks instead whether a carefully engineered validation architecture can stop hallucinations from *reaching the user*. In the words of the project description, a Confident Agent is not one that is always correct; it is one that knows when it should not be trusted.

1.2 Problem statement

Software APIs are a domain in which hallucination is both common and checkable. A model's knowledge of an API is a snapshot of its training data, whereas public APIs keep changing: versions are retired, hosts move and authentication is added. The model has no internal signal that its knowledge is stale. Many APIs, however, publish an authoritative, machine-readable contract, an OpenAPI specification, against which a claim in generated code can be checked without trusting the model. The problem studied here is whether such external evidence, applied by an agent before code is released, measurably reduces the hallucinated API calls that reach a developer, and at what cost in correct answers, time and energy.

1.3 Aim and objectives

The aim is to produce experimentally defensible evidence for or against the effectiveness of such an agent. The objectives follow the empirical workflow of the project description:

1. construct a hallucination benchmark for REST API code generation with an objective, executable answer key (Chapter 4);
2. measure an unmodified baseline model on that benchmark (Chapter 5);
3. design a Confident Agent that validates API claims against external evidence and estimates confidence explicitly (Chapter 6);
4. evaluate the baseline and the agent on the identical benchmark with an independent evaluator (Chapters 7 and 8);
5. analyse the evidence statistically (Chapter 9) and examine the threats to its validity (Chapter 10).

1.4 Contributions

- **A frozen hallucination benchmark** of 1,000 Python tasks over six public REST APIs, generated from pinned, hashed provider specifications with an executable answer key.
- **A Confident Agent** that verifies every API claim in six layers against frozen evidence, scores the evidence with an explicit confidence metric and releases code only through a decision policy with hard constraints.
- **A paired experiment** on 990 tasks under three quantizations, judged by an independent sandboxed evaluator, with a pre-registered analysis plan, exact tests, cluster-bootstrap intervals and a cost analysis in time, tokens, energy and Rand.
- **An architectural finding**: release decisions can be governed by explicit verification constraints rather than by the model's own judgement or by a confidence threshold.

1.5 Structure of the report

The report follows the structure required by the project description. Chapter 2 states the research question and hypotheses, Chapter 3 the application domain, Chapter 4 the benchmark and Chapter 5 the baseline. Chapter 6 presents the Confident Agent, Chapter 7 the experimental methodology, Chapters 8 and 9 the results and their statistical analysis, Chapter 10 the threats to validity and Chapter 11 the conclusions. Implementation details, supplementary analyses and reproduction instructions are in the appendices, and Appendix A summarises the supplementary charts on one page.

2 Research Question and Hypotheses

2.1 Research question

The project description poses the general question: *can an agent-based architecture detect or prevent unacceptable hallucinations before they reach the user?* For the domain of this study it becomes:

> **Research question.** Can an LLM-based code-generation agent that verifies the REST API claims in its own code against frozen, authoritative specifications reduce the unacceptable API hallucinations that reach the user, compared with the same model without verification, while abstaining when verification is inconclusive, and at what cost?

Reaching the user places the harm at the point of delivery: a hallucinated candidate that the agent withholds is not a failure, but one that it delivers is. *Frozen, authoritative specifications* distinguishes the approach from self-verification by the same model, which can rationalise its own errors. *Abstaining* makes refusal a measured outcome rather than a hidden one, and *at what cost* acknowledges that trustworthiness may be bought with time, energy and lost answers.

2.2 Hypotheses

The scientific hypothesis of the project description, that agent-based validation architectures equipped with explicit confidence estimation can significantly reduce unacceptable hallucinations compared with an equivalent baseline model, is tested as H1. Three secondary hypotheses cover the side effects on which any such intervention must also be judged. All four were fixed in an evaluation plan before the main runs.

> **H1 (primary).** Agent-based validation equipped with explicit, evidence-based confidence estimation significantly reduces the proportion of tasks in which code containing an API hallucination (class A, Section 3.2) reaches the user, compared with the equivalent baseline model. Test: exact two-sided McNemar test on paired tasks within each quantization run, $\alpha = 0.05$, Holm-adjusted across the three runs.

- **H2 (correctness is preserved).** The proportion of tasks in which correct code reaches the user does not decrease. A verifier that removed hallucinations by refusing everything would fail H2.
- **H3 (the confidence metric discriminates).** Among final candidates, correct code receives a higher confidence score than hallucinated code (area under the ROC curve above 0.5). The score is not claimed to be a calibrated probability.
- **H4 (the cost is measurable).** The intervention increases time, tokens and energy per prompt; the ratios and their price are estimated rather than tested for direction.

2.3 Outcomes and guiding principles

The agent can succeed in two ways: by detecting and correcting a hallucination, or by withholding code it cannot verify. It can fail in two ways, which are reported separately as the project description requires: **incorrect acceptance** (hallucinated code is delivered) and **incorrect rejection** (correct code is withheld). Abstention is reported as a cost, and a reduction in hallucinations bought only through blanket refusal would count as a negative result.

The investigation follows the three guiding principles of the project description. *No intervention without a baseline*: the same model, settings and prompts are measured without the agent (Chapter 5). *No claim without evidence*: every number, table and chart in this report is computed by scripts from the recorded runs (Appendix J). *No conclusion without discussing validity*: Chapter 10 examines each of the threats the project description names.

3 Application Domain

3.1 Software APIs, REST and OpenAPI

An **application programming interface (API)** is a documented, programmatically accessible interface that software invokes to perform an operation. A **REST API** exposes resources at URLs and manipulates them with the standard HTTP methods (Fielding, 2000). An **OpenAPI** document is a machine-readable contract for a REST API: it declares servers, paths, methods, parameters with their locations and types, request bodies, response schemas and security schemes (OpenAPI Initiative, 2021). A claim about an API in generated code therefore has six parts, which map one-to-one onto the verification layers of Chapter 6: the library imported, the identifier called, the access point (host, path and method), the input contract, the output contract, and the required context such as authentication. Because the contract is authoritative and machine-readable, a claim such as `GET https://api.github.com/orgs/{org}` can be checked deterministically, without executing the code or calling the service.

3.2 Hallucination in generated code

In natural-language generation, a hallucination is generated content that is fluent and plausible but unfaithful to, or unsupported by, the source or the world (Ji et al., 2023; Huang et al., 2025). Code generation exhibits its own forms: invented packages (Spracklen et al., 2025), non-existent functions and wrong API usage (Liu et al., 2024). Patil et al. (2023) reduced hallucinated API calls by grounding a model in retrieved API documentation, which suggests that external evidence is the natural remedy in this domain. This study adopts the working definition: **an LLM hallucination is content that appears plausible and confident but is incorrect, fabricated, or unsupported by verified ground truth.**

An **API hallucination** is a claim in generated code that is verified to be incorrect against an authoritative source. *Cannot be verified* does not mean *false*: a claim can be verified correct, verified incorrect, unverifiable, or not applicable. Only the second is a hallucination; the third is a reason to abstain. The independent evaluator (Section 7.4) classifies the code that reaches the user into the classes of Table 1; Appendix H lists the kinds of error within each class.

Table 1: Classes used to score the code that reaches the user.

Class	Meaning and examples	Counts as
A API hallucination	Breaks the API contract: wrong host, version or base path; undeclared path, method, parameter or field; missing required part or credential; value of the wrong type; reading an undocumented response field; a non-existent `requests` function.	Unacceptable hallucination (primary outcome)
B task-fidelity error	A valid API call that is not the one the task asked for: another declared endpoint, an unacceptable method, a different parameter value or request body.	Secondary outcome
Correct (incl. **C** equivalent)	The expected request, allowing a closed list of equivalences (for example `token X` for `Bearer X`).	Correct
Runtime failure	The code makes no request: syntax error, crash or no call.	Error, not a hallucination
Undetermined	A name a partial specification does not list; resolved by manual review.	Resolved before analysis

3.3 Why external evidence is needed, and scope

A model's knowledge is a snapshot that ends at a cut-off and over-represents whatever was common on the web (Cheng et al., 2024). For APIs this means the model tends to write the interface that was popular in its training data rather than the one that exists today, and nothing in its weights signals that the knowledge is stale (Section 5.3). No amount of fluency or self-reflection repairs this; only evidence from outside the model can. The Confident Agent is therefore anchored to frozen, current specifications rather than to the model's own judgement.

The study is scoped to **Python with the `requests` library**, which gives one ubiquitous HTTP interface and a mature static type checker (Microsoft, 2026), and to **REST APIs with OpenAPI contracts**. Verification is **static**: the agent never executes generated code, because model output may be destructive, a verdict obtained by execution would depend on a live service on a particular day, and execution would need real credentials. The evaluator does execute candidates, but only in an isolated sandbox, as a measuring instrument rather than as part of the agent.

3.4 Key terms in this study

Table 2**:** The terms defined by the project description and how this study realises them.

Term	Realisation in this study
Confident Agent	The Qwen3.5-9B model with the Confident Agent prompt and the `verify_candidate` validation tool, which together decide whether code is delivered, flagged, corrected or withheld (Chapter 6).
Hallucination Benchmark	1,000 Python REST API tasks over six public APIs with an executable answer key (Chapter 4).
Baseline Model	The same model, settings and prompts with a minimal system prompt and no validation (Chapter 5).
Confidence Metric	The Evidence-Weighted Confidence Score: the share of applicable verification weight supported by positive evidence (Section 6.4). It measures evidence coverage, not a probability.
Validation Pipeline	Claim extraction, six verification layers against frozen specifications, the confidence score and the decision policy (Chapter 6).

4 Benchmark Design

4.1 Requirements

The project description requires a benchmark that is representative of the domain, consistently exposes the behaviour under study, permits objective evaluation and is documented well enough to be reproduced. A benchmark that compares two conditions must additionally keep its ground truth fixed between them. The benchmark meets these requirements as follows.

- **Representative:** six widely used public REST APIs that span no authentication, API keys, Bearer tokens and OAuth2, official and documentation-derived specifications, and create, read, update, delete and search operations.
- **Exposes the behaviour:** prompts describe ordinary integration tasks and never reveal the URL, method or parameter names, so the model must supply the API knowledge itself.
- **Objective:** each task carries an exact expected request and its API specification; correctness is decided by comparing requests, not by judging prose.
- **Frozen and hashed:** every provider specification was retrieved on 21 September 2026, stored with its SHA-256 hash and referenced by hash from every task, so the ground truth cannot change between the baseline and agent runs.
- **Safe and reproducible:** no task contains a real credential (tasks read a placeholder from `inputs["credential"]`), and the dataset is generated by a deterministic script; no language model wrote the reference answers.

4.2 APIs and ground-truth evidence

Table 3**:** APIs in the benchmark and the provenance of their evidence (retrieved 21 September 2026).

API	Tasks	Families	Ground-truth source	Tier	Authentication
GitHub REST	250	65	Official OpenAPI, full document (819 paths)	1	Bearer token

API	Tasks	Families	Ground-truth source	Tier	Authentication
Swagger Petstore	150	11	Official OpenAPI 3 document	1	API key / OAuth2
Open-Meteo	150	3	Official OpenAPI (forecast, air quality, marine)	1	None
JSONPlaceholder	150	36	Curated partial specification from the official guide	2	None
DummyJSON	150	23	Curated partial specification from the documentation	2	None
REST Countries v5	150	4	Curated partial specification from the v5 documentation	2	API key (Bearer)

Tier 1 = provider-published specification; tier 2 = researcher-curated partial specification, labelled as such in every file. Total: 1,000 tasks in 142 operation families.

Three APIs publish an official OpenAPI document; GitHub's is frozen in full, which lets the evaluator distinguish an invented path from a real operation that is not the one the task asked for. For the other three no adequate official document exists, so small specifications were built from the cited documentation pages and labelled as researcher-authored subsets. A name that such a specification does not list is not proof that the name does not exist: the evaluator marks it undetermined and a documented manual review resolves it (Section 7.5). REST Countries recently moved to an authenticated v5 interface on a new host; it is included because it is precisely the case in which a model's training data is out of date. Appendix B.1 gives the provenance in detail.

4.3 Tasks, prompts and the answer key

The dataset builder selects documented operations, varies supported parameters and fields, and constructs each record from a checked Python template. Related variants share an **operation family** identifier (141 families among the scored tasks), so that the analysis can allow for their correlation. Each prompt names the API and describes the operation and its inputs, asks for a function `solve(inputs)`, and states how to treat an empty response and an unsuccessful one; these requirements give behaviour checks beyond API correctness (Appendix F gives an example). The 990 scored tasks comprise 494 reads, 232 creates, 197 updates, 37 searches and 30 deletes; 545 use official and 445 curated specifications. Proportions follow what each specification genuinely supports, so results are reported by subgroup rather than assuming a balanced design (Appendix B.2).

The task file is the answer key, and the prompt file sent to both conditions is identified by its hash in every run manifest; neither was modified after the baseline had been run. Before any model output was scored, the evaluator's **self-check** executed all 1,000 reference solutions and classified every one as correct with every behaviour requirement met. A group member also reviewed a stratified sample of 100 tasks, 15–25 per API, against the providers' documentation; all 100 passed every check (Section 7.6). Ten pilot tasks used to develop the agent are excluded from every statistic.

4.4 What the benchmark measures

Every task is answerable and has exactly specified ground truth; there are no trick premises and no tasks for which refusal is the correct answer. The benchmark therefore measures the **natural** rate at which the model writes hallucinated API calls on ordinary requests, which is what a developer would experience, and version drift enters naturally where an API has changed. The price of this choice is that the benchmark cannot reward a correct refusal of an impossible request (Section 10.1).

5 Baseline Evaluation

5.1 Baseline configuration

The baseline is the unmodified model, served exactly as in the intervention: Qwen3.5-9B (GGUF) in LM Studio, at temperature 0.2 without a fixed seed, with reasoning off, a 30,208-token context and at most 4,096 completion tokens per reply. Its system prompt asks only for the requested code in one Python block, and it receives the identical prompt file as the Confident Agent. The experiment was run three times, with the Q4_K_M, Q6_K and Q8_0 quantizations of the same weights, between 30 September and 3 October 2026, on one workstation (Appendix C.1 gives the full configuration).

5.2 Baseline results

Table 4**:** Baseline: class of the code delivered to the user (990 tasks per run, pilot excluded).

Class	Q4_K_M	Q6_K	Q8_0
Correct	247 (24.9%)	365 (36.9%)	341 (34.4%)
A: API hallucination	513 (51.8%)	503 (50.8%)	546 (55.2%)
B: valid call, wrong task	79 (8.0%)	56 (5.7%)	58 (5.9%)
Runtime failure or no code	151 (15.3%)	66 (6.7%)	45 (4.5%)

Between 51% and 55% of baseline answers contained an API hallucination at every quantization, and only a quarter to a third were correct (Table 4). The hallucination rate barely depends on quantization: it reflects what the model knows, not its numerical precision. The lower-precision model pays instead in ordinary programming errors; 128 of Q4_K_M's 150 runtime failures used an input as the wrong type, typically treating the credential string as a dictionary.

5.3 Why the baseline hallucinates

Most baseline hallucinations (1,303 of 1,562 across the three runs) are of a single kind: the wrong host, version or base path. Figure 1 shows which host and version each answer called, against the frozen specification each task pins. Three patterns account for them.

- **Retired versions.** 78% of Petstore answers called the Swagger 2 host `petstore.swagger.io/v2`, which dominates tutorials and training data, instead of the current OpenAPI 3 service; 63% of REST Countries answers called the deprecated `restcountries.com/v3.1` interface.
- **Extrapolated versions.** Told to use "REST Countries v5", which it has never seen, the model grafted the number onto the old URL scheme and invented `restcountries.com/v5.1/...` in 35% of answers; only 4 of 447 answers reached the real host.
- **Sibling confusion.** 45% of DummyJSON answers called JSONPlaceholder, a similar fake-data service, and 67% of Open-Meteo answers sent air-quality or marine requests to the forecast host. Where the model's knowledge is current (JSONPlaceholder, GitHub), the host was almost always right.

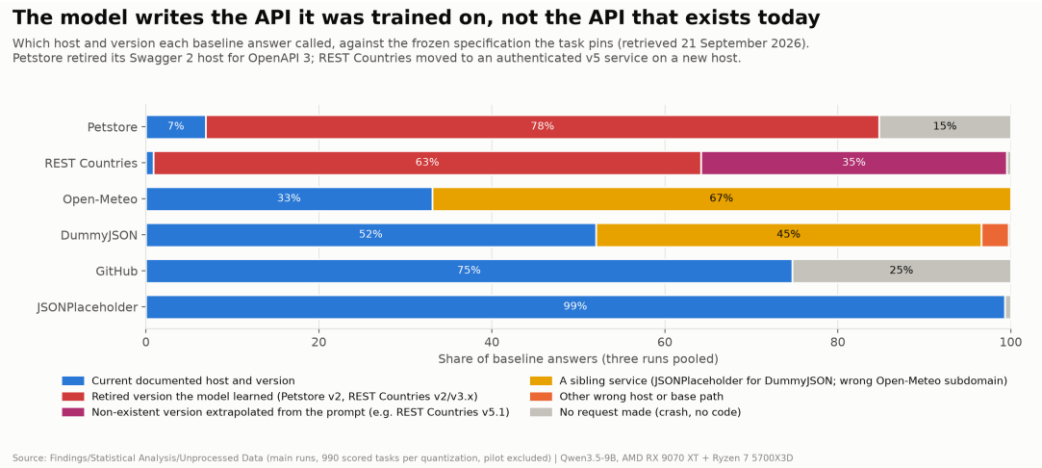


Figure 1: Which API host and version each baseline answer called (three runs pooled). The model reproduces versions from its training data and extrapolates versions it has never seen.

These are errors that a model cannot detect by introspection, since nothing in its weights records that an API has moved, and errors that a frozen specification detects deterministically. They define the problem the Confident Agent must solve.

6 Confident Agent Design

6.1 Design principles

The design rests on one principle: generation alone must not confer authority on an output. It instantiates the six Trust-by-Design patterns of Vorster (2026a): *independent verification* (a deterministic tool, not the model, judges the code), *domain-specific validation* (the evidence is the API's own contract), *evidence fusion* (six layer verdicts combined into one score), *confidence gating* (a policy decides what is released), *deferred commitment* (contradicted code is corrected or withheld, never released) and *provenance preservation* (every decision is recorded with the hashes of the code and the evidence). Appendix D.1 shows where each pattern is realised.

6.2 Architecture

The project description anticipates a coordinating chief agent supported by specialised agents. The Confident Agent keeps this separation of roles but implements the specialised roles, namely API validation, verification against external knowledge and confidence estimation, as a deterministic tool rather than as further language models. A model used as a judge can itself hallucinate; a contract check cannot. The LLM therefore acts as the chief agent, which generates code and corrects it when told precisely what is wrong, while the validation tool `verify_candidate` decides what may be released (Figure 2). The tool is a TypeScript package with a small Python claim extractor, loaded in LM Studio as a tools provider; in the experiment a bridge executes the same implementation for the benchmark harness. The evaluator is a separate program that shares no code with it.

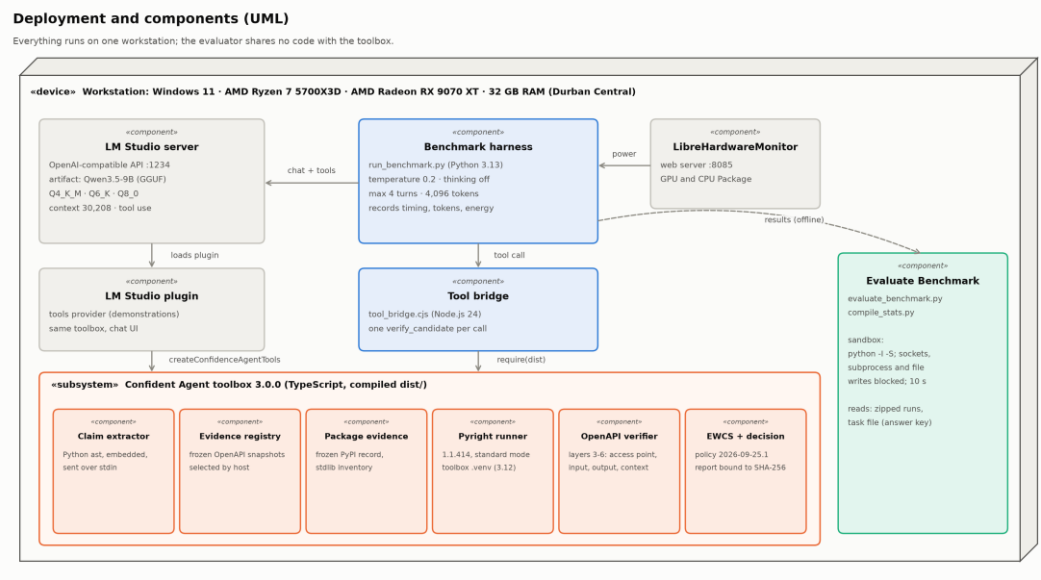


Figure 2**:** Deployment and component view (UML). All components run on one workstation; the evaluator is independent of the toolbox.

Validation is attached **in the loop, through function calling**, so the deployable artefact is what a developer would actually run: a model, a system prompt and one tool (alternatives are compared in Appendix D.2). Because the model then controls the conversation and may skip or repeat steps, three safeguards keep the delivery decision with the tool. The benchmark scores the tool's decision on the last candidate it verified; the tool's report embeds the SHA-256 of the verified code, so an approving report cannot be attached to different code; and every deviation from the prescribed protocol is recorded.

6.3 Verification layers and evidence

Verification begins by turning code into claims. The extractor parses the candidate with Python's `ast` module, without running it, and abstractly interprets the function body; values that exist only at run time become typed holes, so a URL such as `f"https://api.github.com/orgs/{org}"` is matched against OpenAPI path templates. Six layers then check the claims, each against one kind of evidence (layer weights in brackets):

- **L1 Interface (3):** every imported module exists (interpreter module list; frozen package-registry record).
- **L2 Identifier (3):** the Pyright 1.1.414 type checker finds no undefined name, attribute or call in the exact candidate.
- **L3 Access point (4):** the host and base path match a server of the frozen specification, and the path and method match a declared operation.
- **L4 Input contract (2):** required parameters, their locations and types, and request-body properties match the operation.
- **L5 Output contract (1):** every response field the code reads is documented in the success-response schema.
- **L6 Contextual (2):** the operation's security requirements are met by the actual request.

Each layer returns one verdict, namely verified correct, verified incorrect, unverifiable or not applicable, with an evidence tier and a human-readable detail; a contradiction names the closest declared operations, which the model can use to correct its code. The evidence is the same set of frozen, hashed specification snapshots as the benchmark, **selected by the host the candidate itself calls**; the model never supplies or summarises a specification. A host without frozen evidence yields no evidence, so layers 3 to 6 are unverifiable and the code is withheld: an invented host cannot be "verified". Specifications are frozen rather than discovered at run time because a live specification could change between the baseline and agent runs, and because four of the six APIs publish no discoverable specification (Appendices D.3–D.5).

6.4 The confidence metric

Confidence is estimated by external, tool-assisted validation, one of the approaches the project description lists. The **Evidence-Weighted Confidence Score (EWCS)** answers one question: *what proportion of the applicable verification weight is supported by positive evidence?* With A the set of applicable layers, w_l the weight of layer l, t_l the factor of the evidence tier behind its verdict, and c_l = 1 if the layer is verified correct and 0 otherwise:

$$EWCS = \frac{\sum_{l \in A} w_l \cdot t_l \cdot c_l}{\sum_{l \in A} w_l}$$

The weights convert a severity ranking of the layers into numbers with the rank-sum rule (Stillwell, Seaver and Edwards, 1981; Barron and Barrett, 1996): a wrong access point ranks first, because nothing downstream of it can be right. The tier factors discount weaker evidence: 1.0 for a provider-published specification, 0.8 for a curated specification or registry record, 0.5 for a clean type check (the absence of an error is weak evidence) and 0 for no evidence. Because weights and tiers are fixed, the highest attainable score is known in advance: only fully modelled official specifications can reach 0.85, and curated specifications cannot exceed 0.74 (Appendix D.6 gives the rationale, a worked example and the ceilings). The EWCS therefore measures **evidence coverage, not the probability that the code is correct**.

6.5 The decision policy

Figure 3 shows the decision policy. It applies three rules in order, and the first two do not consult the score:

1. **Safety constraint.** If any layer is verified incorrect, the code is not released. On the first candidate the agent may correct the code once and must have the new code verified; on a second candidate it abstains.
2. **Uncertainty gate.** If any applicable layer is unverifiable, failed to run, or had incomplete extraction, the agent abstains.
3. **Thresholds.** Otherwise an EWCS of 0.85 or more delivers the code, 0.60 to 0.85 delivers it with a flag, and a lower score abstains.

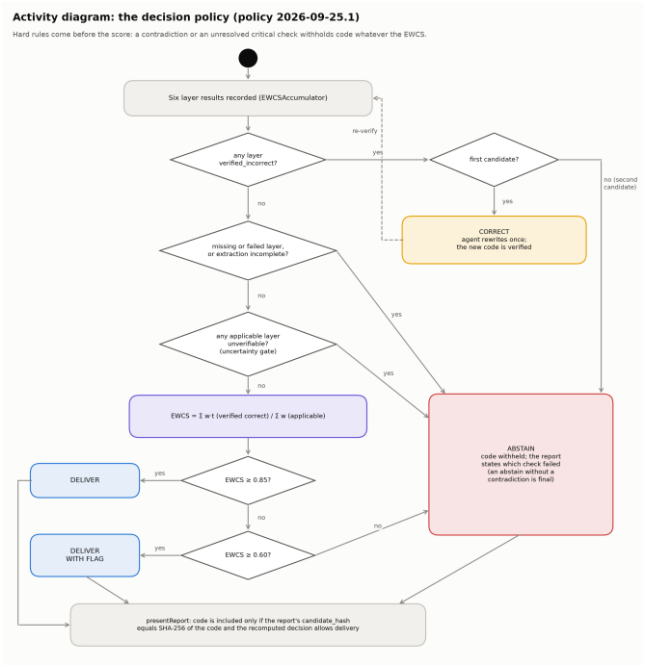


Figure 3**:** Activity diagram (UML) of the decision policy. Hard rules precede the score; the report renderer releases code only when its hash matches the verified candidate.

The design therefore has three distinct parts. The **confidence score** summarises how much evidence supports a candidate. The **hard verification constraints**, the safety constraint and the uncertainty gate, decide whether code may be released at all. The **decision policy** composes them in a fixed order, so the score is consulted only for candidates that the constraints already allow. A candidate that passes the gate has every applicable layer verified and therefore scores at least 0.65; the 0.60 threshold can never bind, and the score in practice decides only whether delivered code carries a flag. Section 8.5 confirms this empirically. A single correction is permitted so that repeated sampling cannot masquerade as correction, and the report renderer recomputes the decision from the per-layer results rather than trusting a score passed to it.

6.6 Validation of the apparatus

Before it was used on model output, the verifier was tested on code whose correctness is known (Table 5). It accepted all 1,000 reference solutions without a single contradiction and contradicted or withheld every one of 3,000 mutants with an invented path, an undeclared method or a wrong host. These are checks of the apparatus, not experimental results.

Table 5**:** Checks of the verifier on known-valid and known-invalid code.

Input	Result
1,000 reference solutions (known valid)	mean EWCS 0.784; 99 deliver, 888 deliver with flag, 13 abstain; **0 contradictions**
1,000 mutants with an invented path	all 1,000 contradicted (correction requested)
1,000 mutants with an undeclared method	all 1,000 contradicted
1,000 mutants with a wrong host	all 1,000 withheld (no evidence for an unknown host)

The 13 abstentions on valid code are documented evidence limits (GitHub request bodies declared with one0f, and one ambiguous path template).

7 Experimental Methodology

7.1 Experimental design

The experiment is a **paired, within-task comparison** (Figure 4). Each of the 990 scored tasks is answered by the same model with the same settings twice, once in the baseline condition and once in the intervention condition. The unit of analysis is the task, and the primary outcome is binary per task and condition: did code containing an API hallucination reach the user?

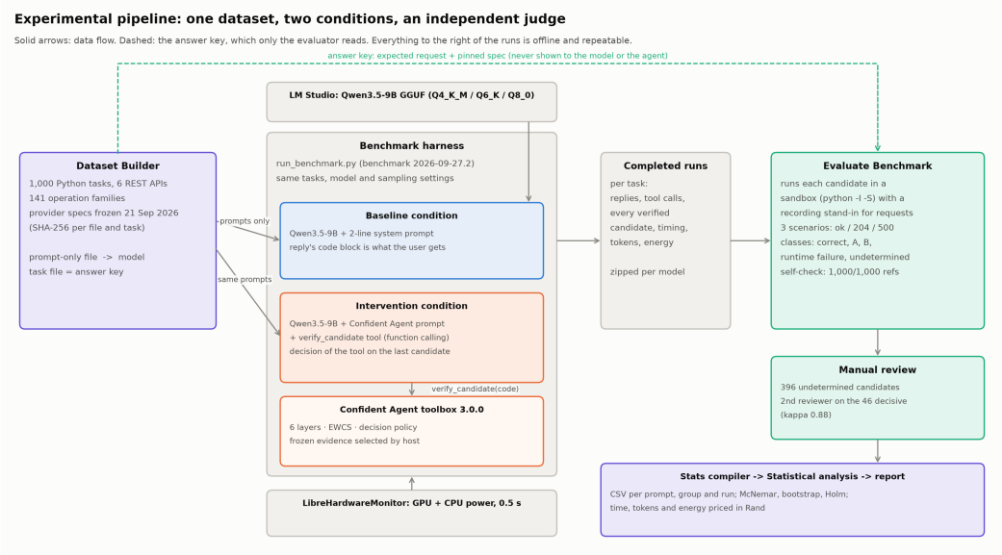


Figure 4**:** Experimental pipeline. The answer key reaches only the evaluator; the model and the agent see the prompt alone.

The whole experiment was run three times, under the Q4_K_M, Q6_K and Q8_0 quantizations, which gives **2,970 task pairs**. These are **not three independent replications** of one configuration. Each run uses the same 990 tasks and prompts and the same model apart from numerical precision, so a task's outcomes are strongly correlated across runs: whether the baseline hallucinated on a task agrees between quantizations with Cohen's $\kappa = 0.69–0.74$, and 412 tasks hallucinated in all three runs while 363 hallucinated in none (Appendix E.10). Within a run each task was sampled once per condition, and tasks are grouped into 141 operation families and six APIs. Each run is therefore tested separately on its 990 pairs; agreement across the runs is treated as evidence of **robustness to quantization** rather than as replication; pooled figures are descriptive; and a cluster bootstrap allows for the family structure.

The intervention is evaluated as a **package**: the Confident Agent prompt (which prescribes the workflow and names the hosts for which evidence is held), the validation tool, one permitted correction, and abstention. Each condition generates its own code, because the prompt also shapes the first answer. The experiment therefore measures the effect of the **combined architecture**; it was not designed to isolate each component, and Section 8.3 reports what the recorded data can and cannot say about them.

7.2 Procedure

The benchmark harness sends each prompt to LM Studio. In the **baseline**, the model's single reply is stored, and the Python code block it contains is what the user would receive. In the **intervention**, the

model may call `verify_candidate`; the harness executes the call, stores the candidate byte for byte with the tool's full result, and returns the result to the model. The prompt instructs the model to correct its code once if, and only if, a layer is verified incorrect, and then to reply with the tool's report verbatim; if the model has not replied after four turns, one fixed message with tools disabled asks for the report. What reaches the user is the code the tool delivered on the last verified candidate. Every message, tool call, candidate and timing is saved.

7.3 Measurements

For every task and condition the study records the class of the delivered code (the primary and secondary outcomes), whether code was withheld, incorrect acceptance and incorrect rejection, the verifier's decision, score and layer states, corrections and protocol deviations, and three behaviour checks (the expected return value, `None` for an empty response, and an exception for a server error). Cost is measured as wall-clock time, prompt and completion tokens, and GPU and CPU package energy sampled every half second, which is converted to Rand at the Durban Central electricity tariff of R4.1109 per kWh including VAT (eThekwini Municipality, 2026). Appendix C.3 defines each measure.

7.4 Independent evaluation

An experimental grader must not rely on the agent's own verdict, or the agent's blind spots would become the experiment's. The evaluator therefore shares no code with the toolbox and uses a different method (Figure 5): it executes each candidate in an isolated Python process with the task's test inputs and a recording stand-in for `requests` that sends nothing. This yields the exact requests the code would send, including those built in loops and branches, and every response field it reads. Each candidate runs against a documented success response, an empty response and a server error, which give the behaviour checks, and its requests are compared with the expected request and the pinned specification. Before scoring any model output, the evaluator scored all 1,000 reference solutions as correct.

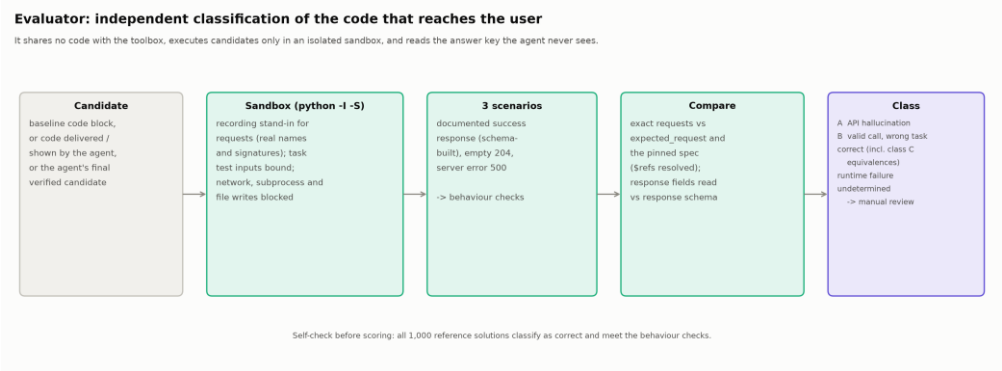


Figure 5: The independent evaluator: sandboxed execution, three response scenarios and comparison with the answer key.

7.5 Manual review of undetermined candidates

For the three curated specifications, a name the specification does not list cannot be classified automatically. Across the three runs there were **396 such undetermined candidates**: 46 had reached the user and could change the primary result, and 350 had been withheld. Each open question was answered once against the provider's documentation and, where it was silent, against the live API with harmless read requests, and recorded as an explicit rule applied to every candidate asking the same

question (Appendix I). The first review was AI-assisted; a group member then independently reviewed the 46 decisive candidates, and the two reviews agreed on 43 (Cohen's κ = 0.88). The three disagreements were settled as correct by the group, which changes no hallucination figure.

7.6 Human audits of the answer key and the evaluator

Passing all 1,000 reference solutions shows that the evaluator accepts correct code; it does not show that the evaluator labels errors correctly, and the answer key was written by the same group. Two human audits address this. In the **answer-key audit**, a group member checked a stratified sample of 100 tasks (15–25 per API) against the providers' documentation on six criteria: the source contract, an unambiguous prompt, the reference code, the test case, alternative valid calls and the difficulty label. All 100 tasks passed every criterion. In the **evaluator audit**, two group members independently classified a stratified random sample of 102 of the evaluator's decisions from the Q6_K run (52 baseline answers and 50 delivered by the agent, covering every class) and then adjudicated their differences (Table 6).

Table 6: Evaluator audit: agreement on 102 sampled decisions from the Q6_K run (Cohen's κ in brackets).

Comparison	n	Same class	Same primary outcome (A or not)
Reviewer 1 vs evaluator	102	96.1% (0.94)	100% (1.00)
Reviewer 2 vs evaluator	102	78.4% (0.65)	89.2% (0.73)
Reviewer 1 vs reviewer 2	102	82.4% (0.72)	89.2% (0.73)
Adjudicated class vs evaluator	84	95.2% (0.92)	100% (1.00)

The completed audit is `Findings/EvaluateBenchmark/reviews/audit_reviews.csv`; the figures are computed by `audit_analysis.py`. 18 decisions on which the reviewers disagreed await adjudication.

Reviewer 1 agreed with the evaluator on every decision about the primary outcome. The four class differences, which the adjudication upheld, are programs that send exactly the right request but always return `None`; the reviewers counted them as task-fidelity errors, whereas the evaluation plan scores the request and checks the return value separately as behaviour (Section 8.4). Reviewer 2 differed from the evaluator on 22 decisions. Seven of these reflect a different rule rather than a different judgement: Reviewer 2 classified programs that crash before sending a request by the request they would have sent, whereas the evaluation plan classifies them as runtime failures. Four concern updates sent with PUT where the task's acceptable methods include PUT; four are the programs that always return `None`; and the remaining seven are individual judgements on request bodies, headers and parameters, four of them for Petstore. Every adjudicated decision agrees with the evaluator on the primary outcome; the 18 unresolved disagreements are a residual threat to the evaluator's validity (Section 10.2).

7.7 Pre-registration and statistical methods

To prevent scoring rules from being chosen after seeing results, the classes, outcomes and statistical tests were fixed in an evaluation plan on 27 September 2026, before the main runs, and every later change was logged with its reason (Appendix C.6). The primary outcome is paired and binary, so the primary test is the exact two-sided McNemar test within each run (McNemar, 1947), at $\alpha = 0.05$ with Holm's correction across the three runs (Holm, 1979). Effect sizes are differences in proportions with 95% percentile bootstrap intervals (10,000 resamples) over tasks and, to allow for related prompts,

over the 141 operation families (Efron and Tibshirani, 1993). Subgroup rates carry Wilson intervals and are descriptive (Wilson, 1927); costs are compared with the Wilcoxon signed-rank test (Wilcoxon, 1945); the confidence metric is assessed by the area under the ROC curve; and agreement by Cohen's κ (Cohen, 1960). Appendix C.7 lists the full plan.

8 Experimental Results

All figures in this chapter are computed over the 990 scored tasks per run, after the manual review; no undetermined candidates remain. Each run is reported separately, and pooled totals, which repeat the same tasks, are descriptive (Section 7.1). Supplementary analyses are in Appendix E, and Appendix A summarises the supplementary charts.

8.1 Headline results

Table 7: Headline outcomes per run (990 tasks each; baseline → agent).

Outcome	Q4_K_M	Q6_K	Q8_0
Hallucination (A) reaching the user	51.8% → 1.6%	50.8% → 2.3%	55.2% → 2.0%
Tasks repaired / regressed (A)	505 / 8	490 / 10	529 / 3
Correct code reaching the user	24.9% → 50.9%	36.9% → 50.4%	34.4% → 51.5%
Abstention (no code shown)	37.6%	39.5%	38.7%
Incorrect acceptance / incorrect rejection	16 / 21	23 / 20	20 / 28

In every run, the share of tasks in which hallucinated code reached the user fell by 48.5–53.1 percentage points, to about 2%, and the share receiving correct code rose by 13.5–26.0 points, to about 51% (Table 7). The agent's final accuracy is almost the same at every quantization even though the baselines differ: the lower-precision model gains most because the agent also repairs the ordinary coding errors in which its baseline was weakest. The price is that 38–40% of tasks end with no code at all.

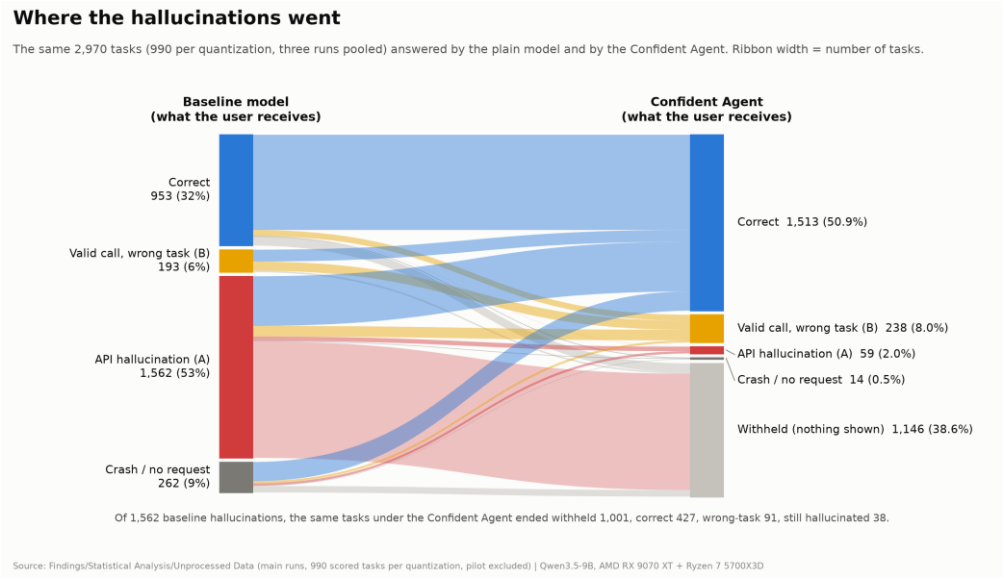


Figure 6: Each task's outcome under the baseline (left) and under the Confident Agent (right), three runs pooled; ribbon width is the number of task-runs.

Figure 6 follows each task from its baseline outcome to its intervention outcome. Of the 1,562 baseline hallucinations across the three runs, the agent withheld the code in 1,001, delivered correct code in 427, delivered a valid call that did not match the task in 91, and still delivered a hallucination in 38. Of the 953 baseline answers that were correct, it delivered correct code in 819, withheld 76, delivered a different valid call in 51 and turned one into a hallucination. About two-thirds of the hallucinations it prevented were thus prevented by **withholding** code and one-third by **correcting** it. Withholding is the intended behaviour when evidence contradicts a candidate, but it is not an answer, so the 1,146 withheld tasks are counted as a cost rather than a success.

8.2 Incorrect acceptance and incorrect rejection

Treating the verifier as a classifier of the agent's final candidate (positive = hallucinated, predicted positive = withheld), its recall is **94–96%**: incorrect acceptance occurred in only 16–23 tasks per run. Its precision is **86–92%**: most of what it withholds really is hallucinated. Incorrect rejection, correct final candidates withheld, occurred in 20–28 tasks per run, about 2–3% of tasks (Appendix E.4). Both kinds of error are therefore small relative to the effect, and abstention is driven overwhelmingly by code that was genuinely wrong. When task-fidelity (class B) errors are also counted as positives, recall falls to 76–80%, because the verifier checks the API contract rather than the task (Section 8.4).

8.3 Where in the architecture the reduction happens

Because the intervention is a package (Section 7.1), the headline shows what the combined architecture achieves. Table 8 locates where the reduction occurs, using the one intermediate observation the benchmark records: the **first candidate** the model submitted for verification. It was written under the Confident Agent prompt, including the list of hosts with evidence, but before any verification feedback, and it is scored by the same evaluator and review decisions as the delivered code.

Table 8: Class A code at three stages: the baseline answer, the agent's first candidate (before verification feedback) and the code the agent delivered.

Stage	Q4_K_M	Q6_K	Q8_0
Baseline answer: class A	51.8%	50.8%	55.2%
Agent's first candidate: class A	48.9%	46.7%	47.9%
… counting undetermined as A	52.8%	54.5%	56.1%
Agent's delivered code: class A	1.6%	2.3%	2.0%
Wrong host, version or base path (tasks)	449 → 256 → 1	408 → 178 → 0	446 → 167 → 0
Other kinds of hallucination (tasks)	58 → 209 → 15	87 → 182 → 23	93 → 203 → 20

The last two rows count tasks at the three stages: baseline → first candidate → delivered; a task can have both kinds. Undetermined first candidates (names a curated specification does not list) were not manually reviewed. Computed by `Findings/EvaluateBenchmark/stage_analysis.py`.

The prompt on its own barely changed *how often* the model hallucinated: 46.7–48.9% of first candidates were class A (52.8–56.1% if the unresolved cases are counted as hallucinations), against 50.8–55.2% of baseline answers. It changed *which* hallucination the model made. Told the current hosts, the model wrote a wrong host far less often (in 167–256 tasks per run, against 408–449), but invented paths and parameters on the right host instead, so hallucinations other than a wrong host rose from 58–93 to 182–209 tasks per run. The fall to 1.6–2.3% happens between the first candidate

and the delivered code, where verification, correction and withholding act; there, hallucinations other than a wrong host fell to 15–23 tasks per run, 74–79% below the baseline.

This is a **decomposition by stage, not an ablation**. The first candidate was written by a model that had been told its code would be verified, so it does not show what the prompt would achieve without the tool; the correction step uses the tool's evidence, so verification and correction cannot be separated; and no condition was run with the tool but the baseline prompt, or with the prompt but no tool. The separate contributions of contextual prompting, verification, correction and abstention therefore remain unmeasured. The data support a narrower claim: the combined architecture works, and the reduction coincides with the verification and release stage rather than with the model's first answer.

8.4 Results by API, and where the intervention does not succeed

APIs the model knows well improve the most: GitHub rises from 26–63% correct to 75–84%, and DummyJSON from 30–59% to 67–74%, with no hallucination reaching the user in any run (Appendix E.2). Three outcomes, however, qualify the headline as much as they support it.

- **REST Countries: safe, but no useful answer.** The baseline hallucinated on 99–100% of REST Countries tasks, and the agent delivered nothing for 99–100% of them. Even with the current host named in its prompt, the model invented v5 paths (all 258 withheld candidates reviewed used non-existent paths), and the verifier correctly refused them, at a cost of 7.1 hours of computation across the three runs. The architecture prevented unsupported code from being released, but it cannot supply code that the model does not know how to write: for this part of the domain it provides safety without usefulness.
- **Residual hallucinations.** 57 of the 59 hallucinations still delivered are Petstore tasks, mostly request bodies that place a run-time value where the schema expects a string. The verifier deliberately treats run-time values as opaque, which is what makes `solve(inputs)` code verifiable at all, so it cannot see which value lands in a field (Appendix E.3).
- **Conformance to the API is not the user's task.** The verifier checks the API contract, not the user's intent, so a valid call to the wrong endpoint passes by design. Task-fidelity (class B) errors in delivered code did not fall but rose, from 193 to 238 across the three runs. Only 19–23% of the agent's delivered programs also met all three behaviour checks, mostly because few returned `None` for an empty response. Requiring the right request *and* all three behaviours, the agent delivered fully correct code for only 10.3–11.3% of tasks, against 2.1–6.5% for the baseline (Appendix E.8).

A clean verification report is therefore evidence about API claims only, which is what the toolbox's report tells the user. Throughout this report, "correct code" means the correct request, not a program that does everything the task asked.

8.5 The confidence metric and the decision policy

The EWCS separates correct from hallucinated final candidates with an area under the ROC curve of **0.932–0.943** (Appendix E.5). Much of this discrimination is inherited from contradictions: hallucinated candidates cluster near a score of 0.3, the value left when the access point is contradicted and the layers that depend on it cannot be matched, which is the same contradiction the safety constraint acts on. The score is not calibrated either. Candidates scoring 0.85 or more were correct only 66% of the

time, less often than those in the flagged band (72%), because only Petstore can reach the top band, while only 3 of 738 candidates scoring below 0.60 were correct.

To test whether any result depends on the score, the final candidate of every task was re-scored under the five weighting and threshold schemes fixed in the evaluation plan, and once more with the uncertainty gate removed (Table 9). Every verification stored its per-layer states and tiers, so the re-scoring is exact: under the scheme actually used it reproduces the tool's recorded decision for all 2,970 task-runs.

Table 9: Sensitivity analysis: outcomes on the final verified candidate (three runs pooled, 2,970 task-runs).

Scheme	Weights L1–L6	Thresholds	Deliver / flag	Abstain	Hallucinations delivered	Correct delivered
A. Proposed (used)	3,3,4,2,1,2	0.85 / 0.60	166 / 1,658	1,146	59	1,513
B. Equal weights	1,1,1,1,1,1	0.85 / 0.60	212 / 1,612	1,146	59	1,513
C. Endpoint-heavy	2,2,6,2,1,2	0.85 / 0.60	1,072 / 752	1,146	59	1,513
D. Strict thresholds	3,3,4,2,1,2	0.90 / 0.70	0 / 1,824	1,146	59	1,513
E. Lenient thresholds	3,3,4,2,1,2	0.80 / 0.50	1,072 / 752	1,146	59	1,513
F. Gate removed (exploratory)	3,3,4,2,1,2	0.85 / 0.60	166 / 1,919	885	**224**	1,579

Under all five planned schemes the withheld tasks, the hallucinations delivered and the correct code delivered are identical; the weights and thresholds only move tasks between *deliver* and *deliver with flag*. Removing the uncertainty gate, by contrast, would deliver 261 more candidates, 165 of them hallucinations and only 66 correct. This is the study's central architectural finding: **in this design the confidence score is informative, but it does not decide release; the hard verification constraints do.** The EWCS explains delivered code to the user, the safety constraint and the uncertainty gate provide the protection, and the decision policy is the explicit ordering that guarantees the score can never override them. Tuning weights or thresholds would not change what is withheld; changing what the gate treats as unverifiable would.

8.6 Correction and agent behaviour

The agent attempted a correction in 403–444 tasks per run. When the verifier named the contradicted claim and the closest declared operations, the model repaired a hallucinated first candidate in 121–150 tasks per run, and about as often produced another hallucinated candidate, which was then withheld; self-correction therefore succeeds about a third of the time (Appendix E.7). The model frequently departed from the prescribed protocol: 34–39% of tasks reached the four-turn limit, about 30% re-verified identical code, and only 53–54% followed the prompt exactly. These deviations cost time but never let unverified code through: **in no task of any run did the user see code the tool had not approved**. This is the practical argument for keeping the delivery decision in the tool rather than in the model.

8.7 Cost of the intervention

Table 10**:** Cost of the intervention (990 tasks per run; baseline → agent; electricity at R4.1109/kWh including VAT).

Measure	Q4_K_M	Q6_K	Q8_0
Mean time per prompt	5.8 → 36.9 s	6.2 → 38.3 s	7.4 → 48.4 s
Energy per prompt (GPU + CPU)	0.48 → 2.56 Wh	0.53 → 3.35 Wh	0.59 → 3.62 Wh
Tokens per prompt (input + output)	530 → 21,420	506 → 22,771	504 → 23,058
Electricity per 1,000 prompts	R1.95 → R10.51	R2.20 → R13.78	R2.42 → R14.87
Time per correct answer delivered	23 → 73 s	17 → 76 s	21 → 94 s
Extra electricity per hallucination prevented	1.70c	2.39c	2.34c

Measured energy covers the GPU and CPU packages only; wall-socket energy is higher.

Every prompt costs more under the intervention (paired Wilcoxon $p < 10^{-160}$ in all 18 comparisons): about 6.2–6.6 times the time, 5.4–6.3 times the measured energy and 40–46 times the tokens (Table 10). The verification itself is cheap (2.3–9.6 s per prompt); the cost comes from the conversation, which re-sends its growing context, including the six-layer report, on every turn. About half of the agent's processing time is spent on tasks whose final candidate is hallucinated and withheld, which is well spent from a safety standpoint but buys no deliverable. At local electricity prices the cost is about one cent per verified prompt; the real price is latency and, on a pay-per-token service, tokens (Appendix E.9). With the verifier, accuracy is almost independent of quantization, so the smallest model, Q4_K_M, gives the best value.

9 Statistical Analysis

9.1 Primary hypothesis

The primary outcome is paired and binary, so McNemar's test is appropriate: it depends only on the discordant pairs, tasks in which the baseline hallucinated and the agent did not (b), and the reverse (c) (McNemar, 1947). The exact binomial version is used. Two 95% intervals are reported for each difference: a bootstrap over tasks, and a cluster bootstrap over the 141 operation families, which does not assume that tasks are independent.

Table 11**:** Primary test: class A code reaching the user, baseline versus Confident Agent (990 paired tasks per run).

Run	Baseline	Agent	Difference	95% CI (tasks)	95% CI (families)	b / c	Holm p
Q4_K_M	51.8%	1.6%	−50.2 pp	[−53.3, −47.0]	[−63.2, −34.6]	505 / 8	1.7×10^{-137}
Q6_K	50.8%	2.3%	−48.5 pp	[−51.7, −45.3]	[−62.1, −32.7]	490 / 10	1.5×10^{-130}
Q8_0	55.2%	2.0%	−53.1 pp	[−56.3, −50.0]	[−65.7, −37.9]	529 / 3	1.1×10^{-152}

H1 is supported in each run. The null hypothesis of no difference is rejected with Holm-adjusted p-values below 10^{-129}, and even the conservative family-level intervals exclude zero by more than 30 percentage points. For every task the intervention made worse, it repaired between 49 and 176.

9.2 Secondary hypotheses

H2 is supported: correct code reaching the user did not merely survive the intervention but increased significantly in every run, by 13.5–26.0 percentage points (family-level intervals all above +7.7 points; $p \leq 6.4 \times 10^{-20}$). Class A or B code reaching the user fell from 56.5–61.0% to 9.4–11.1% ($p < 10^{-106}$), although class B alone did not fall (Section 8.4). **H3 is supported**: the EWCS discriminates correct from hallucinated final candidates with an AUC of 0.93–0.94, but it is not calibrated (Section 8.5). **H4 is quantified**: every cost measure rises in all 18 paired comparisons, with rank-biserial correlations of at least 0.995 and narrow ratio intervals (for example, time ×6.36 [6.13, 6.58] at Q4_K_M). Appendix E.11 gives the secondary results in full.

9.3 Interpreting the significance tests

Each p-value is a within-run test on 990 paired tasks. Because the three runs repeat the same tasks with the same model at different precision (Section 7.1), they are **not three independent confirmations**, and the 2,970 pairs are not 2,970 independent observations. Holm's correction controls the chance of a false positive across the three tests, but it does not turn them into independent evidence; what the three runs show is that the effect survives a change of quantization almost unchanged. Within a run, tasks in the same operation family are related: the family-level intervals are four to five times wider than the task-level ones, yet remain far from zero. Each task was sampled once per condition, and repeat runs with identical settings changed the delivered or withheld status of 11–14% of tasks while reproducing the aggregate rates within about two points (Appendix E.10), so the conclusions rest on aggregate paired comparisons rather than on individual tasks. Finally, the population to which the tests generalise is tasks like these, for this model, on these six APIs; generalising further rests on argument, not on these tests (Section 10.6).

9.4 Robustness checks

- **Pilot tasks included** (1,000 tasks): 51.8% → 1.6%, 50.7% → 2.3%, 55.0% → 2.0%; all $p < 10^{-130}$.
- **Undetermined candidates**: none remain after the manual review; before it, counting them all as hallucinations or all as correct moved the rates by at most 1.5 points and changed no conclusion.
- **Hallucinations other than a wrong host** fell by 74–79% from baseline to delivered code, so the result does not rest on the host list in the prompt (Section 8.3).
- **Weighting schemes**: identical withheld sets and outcomes under all five planned schemes (Section 8.5).
- **Repeat runs** with identical settings reproduce the headline rates within about two points (Appendix E.10).

10 Threats to Validity

Following the project description and the classification of threats in experimental software engineering (Wohlin et al., 2012), this chapter examines benchmark bias, evaluator bias, overfitting, model drift, confidence calibration, the internal validity of the comparison and the generalisability of the results, and then the practical engineering trade-offs. Appendix C.8 summarises the threats and their mitigations.

10.1 Benchmark bias

- **Hallucination means contract violation.** Class A measures conformance to a frozen specification, not whether a live service would accept the request. Some APIs silently ignore unknown parameters, so a class A candidate may still return data; it does not, however, do what the code claims, which is the harm targeted.
- **Ground truth is partly researcher-authored.** Three of the six specifications are curated partial contracts. The answer-key audit found no errors in a stratified sample of 100 tasks, and the evaluator's self-check passed all 1,000 reference solutions, but the audit had one reviewer and covered a tenth of the tasks, so it reduces rather than removes this threat.
- **Composition follows the specifications.** Reads dominate (494 of 990 tasks), deletes are rare (30) and the difficulty heuristic labels only three tasks hard, so pooled rates weight APIs and operations unevenly. Results are therefore also reported by subgroup.
- **Only answerable tasks, narrowly checked.** The benchmark contains no ambiguous or impossible requests, so it cannot measure whether the agent correctly refuses a task that should be refused, and three behaviour checks do not establish that a program is fully correct.

10.2 Evaluator bias

- **Same team.** The same group, with AI assistance, built the agent and the evaluator. They are separated by code (no shared modules), by method (sandboxed execution versus static analysis) and by data (only the evaluator reads the answer key), and the evaluator was validated on the reference solutions. In the two-reviewer audit of 102 of its decisions, every adjudicated decision agreed with it on the primary outcome, but the reviewers agreed with each other only moderately ($\kappa = 0.72$), partly because they applied different rules to crashing programs, and 18 disagreements await adjudication (Section 7.6). The audit covers one run, and its sample is small for estimating rare errors.
- **AI-assisted manual review.** The first review of the 396 undetermined candidates was AI-assisted. A human second review of the 46 candidates that could change the primary result agreed on 43 ($\kappa = 0.88$), and every decision is recorded with its evidence; the 350 withheld candidates have the first review only. Settling the three disagreements as correct relaxed the closed list of equivalences for three cases and changed no hallucination figure.

10.3 Overfitting

- **Development tasks.** The ten pilot tasks used to develop the agent overlap the benchmark; they are excluded from every statistic, and including them changes no result (Section 9.4).
- **Parameters.** The layer weights, tier factors and thresholds were fixed before the main runs, and the sensitivity analysis shows that no outcome depends on them (Section 8.5).
- **One rule informed by benchmark data.** One verifier rule (undeclared request-body fields are unverifiable rather than accepted) was changed after inspecting the first benchmark run. The change is disclosed, applies to every reported run and made the verifier stricter; it is nonetheless a design choice informed by the benchmark.

- **Evidence for these APIs.** The agent holds frozen evidence for exactly the benchmark's APIs and its prompt names their hosts. Its performance on any other API depends on a specification having been frozen for it.

10.4 Model drift

- **Model and runtime.** The model weights and settings were pinned, but LM Studio's GPU runtime updated itself during the project and its version is not recorded in the run manifests. Repeat runs with identical settings reproduce the headline rates within about two points (Appendix E.10).
- **Sampling.** Without a fixed seed, 11–14% of tasks changed between delivered and withheld from one run to the next; individual task outcomes are noisy, while aggregate rates are stable.
- **API drift.** The specifications were frozen on 21 September 2026, and the live APIs will continue to change. Frozen evidence will in time become stale in the same way as the model's knowledge, so a deployment would need to monitor its specifications against their live versions.

10.5 Confidence calibration

The EWCS is not calibrated: candidates scoring 0.85 or more were correct only 66% of the time, less often than those in the flagged band, and its high discrimination is largely inherited from the contradictions on which the safety constraint already acts (Section 8.5). The score must therefore not be read as a probability of correctness. Because the design does not rely on the score for protection, the miscalibration affects only whether delivered code carries a flag, not what is withheld; a user should nevertheless treat the flag as a statement about evidence coverage only.

10.6 Internal validity of the comparison

- **A combined intervention.** Prompt, tool, correction and abstention were evaluated together. The stage decomposition (Section 8.3) locates the reduction at verification and release, but it is not an ablation, so the contribution of each component is not established.
- **Independent samples per condition.** Each condition generated its own code, so answers differ by sampling as well as by treatment. Pairing by task, and effects far larger than the two-point run-to-run variation, limit this threat.
- **Model-controlled protocol.** The model deviated from the prescribed protocol in 46–47% of tasks, but the tool's decision on the last verified candidate determines what reaches the user and no unapproved code was ever shown, so deviations affect cost rather than the primary outcome.
- **Related observations.** The same tasks appear in every run and related tasks share operation families; tests are therefore conducted within each run, intervals are also computed over families, and the runs are treated as a robustness check rather than as replications (Section 9.3).

10.7 Generalisability

The results come from one model family at one size (Qwen3.5-9B), six popular public APIs (two of them fake-data services that the model knows unusually well and one unusually new), Python with `requests`, single-request tasks, one workstation and one electricity tariff. Larger models, which hallucinate less and follow tool protocols better, could shift both the baseline rate and the cost. Enterprise and private APIs, APIs without any specification, other languages and HTTP clients, sessions, asynchronous code and multi-step workflows are outside the scope of the toolbox, and for an unknown host the agent always abstains. The design itself is API-agnostic, but its benefit is conditional on frozen evidence being available.

10.8 Practical engineering trade-offs

The improvement in trustworthiness is bought with latency, tokens, energy, abstention and complexity. Each prompt takes about six times as long (37–48 s against 6–7 s) and processes 40–46 times as many tokens; at local electricity prices this costs about one cent per verified prompt, but on a pay-per-token cloud model the token increase would dominate. The agent withholds code for 38–40% of requests, and for almost every REST Countries request, so a developer often receives "could not verify" instead of code. Operating the agent also requires frozen, maintained specifications for every API it supports, curated specifications where providers publish none, a type checker and a claim extractor. Whether the trade is worthwhile depends on the cost of a hallucinated integration reaching production, which in most settings far exceeds a minute of waiting.

11 Conclusion

11.1 Answer to the research question

This study asked whether an agent that verifies the REST API claims in its own code against frozen, authoritative specifications can stop API hallucinations from reaching the user, and at what cost. For the combined Confident Agent architecture of prompt, verification, one correction and abstention, judged by an independent evaluator on a frozen benchmark whose answer key and evaluator were both audited by group members, the answer is yes. In each of three runs on the same 990 tasks, hallucinated code reaching the user fell from 50.8–55.2% to 1.6–2.3% (exact McNemar, Holm-adjusted $p < 10^{-129}$), while correct code reaching the user rose from 24.9–36.9% to 50.4–51.5%. The scientific hypothesis of the project description is supported for this architecture, this model and these APIs.

11.2 Principal findings

1. **The model writes the API it was trained on.** Its hallucinations are dominated by retired and extrapolated versions and by confusion between similar services. These errors are invisible to the model and obvious to a pinned specification, which is why external evidence, rather than self-reflection, is the appropriate remedy.
2. **The combined architecture works; its components were not separated.** The agent's first candidates, written under its prompt, hallucinated almost as often as the baseline (46.7–48.9% against 50.8–55.2%): naming the current hosts changed which hallucination the model made rather than whether it made one. The reduction coincides with verification and release, but

without an ablation the separate contributions of prompting, verification, correction and abstention are not established.

3. **Release is governed by explicit constraints, not by the confidence score.** The EWCS discriminates well (AUC 0.93–0.94) but measures evidence coverage, is not calibrated, and in this design decides only whether delivered code carries a flag. The safety constraint and the uncertainty gate decide what is withheld: every planned weighting scheme withheld the same code, and removing the gate would have let almost four times as many hallucinations through. The decision policy orders the two so that the score can never override a constraint.
4. **Safe is not the same as useful, and conformant is not the same as correct.** The agent withheld code for 38–40% of tasks and for almost every REST Countries task, where it offers safety but no answer. Passing verification shows conformance to the API contract, not that the code does what the user intended: task-fidelity errors did not fall, and only about one task in ten received code that met every check.
5. **Trust has a price.** The agent takes about six times as long and processes 40–46 times as many tokens; locally this is about one cent of electricity per prompt, while the real costs are latency and, on a paid service, tokens.

11.3 Limitations and future work

The residual hallucinations come from values known only at run time; the verifier does not check whether a valid call is the one the task wanted; the model repairs its own code only about a third of the time; and only APIs with frozen specifications can be verified. These limits point to the next steps: an ablation that runs the prompt without the tool and the tool with the baseline prompt; binding representative inputs during verification; task-level checks of the intended operation; a shorter conversation and fewer correction rounds to reduce cost; adjudicating the remaining audit disagreements and extending the audits to more tasks, more reviewers and every run; repeating the experiment with seeded samples and other models; adding ambiguous and impossible tasks so that correct refusal can be measured; and monitoring frozen specifications for drift against their live versions.

11.4 Closing remarks

A Confident Agent, the project description states, is not one that is always correct but one that knows when it should not be trusted. The agent studied here is wrong far less often than the model it wraps, and when it cannot establish that its code is right, it says so instead of handing the code over. That behaviour does not come from the model becoming more humble, nor from a well-chosen confidence threshold. It comes from explicit constraints in the architecture that decide what may be released: trust by design rather than by declaration (Vorster, 2026b).

AI Disclosure

In line with the academic-integrity requirements of the project description (University of KwaZulu-Natal, 2026, §A.4), the group acknowledges the following use of generative AI.

- **Design discussions.** DeepSeek and ChatGPT were used to explore the API domain and to discuss dataset and verifier designs. The conversation transcripts are retained.
- **Software.** Claude (Anthropic), used through Claude Code, assisted in writing, testing and debugging the dataset builder, the Confident Agent toolbox, the benchmark harness, the evaluator, the statistics compiler and the analysis and diagram scripts.
- **Review.** The first manual review of the 396 undetermined candidates was AI-assisted (Claude), with every rule and decision recorded; a group member independently reviewed the 46 decisive candidates (Section 7.5).
- **This report.** The report was drafted and revised with AI assistance (Claude) from the group's code, data and evaluation plan. Every number, table and chart is generated by scripts in the submission from the recorded experimental data.

All research decisions, including the domain, the hypotheses, the evaluation plan and its changes, were discussed and taken by the group, and the group takes responsibility for the content of this report.

References

Barron, F.H. and Barrett, B.E. (1996) 'Decision quality using ranked attribute weights', *Management Science*, 42(11), pp. 1515–1523.

Cheng, J., Marone, M., Weller, O., Lawrie, D., Khashabi, D. and Van Durme, B. (2024) 'Dated data: Tracing knowledge cutoffs in large language models', *Conference on Language Modeling (COLM)*. arXiv:2403.12958.

Cohen, J. (1960) 'A coefficient of agreement for nominal scales', *Educational and Psychological Measurement*, 20(1), pp. 37–46.

DummyJSON (2026) *DummyJSON documentation: products, users, posts, todos*. Available at: https://dummyjson.com/docs (Accessed: 21 September 2026).

Efron, B. and Tibshirani, R.J. (1993) *An Introduction to the Bootstrap*. New York: Chapman & Hall.

eThekwini Municipality (2026) *Electricity account for the test site, Durban Central, September 2026: energy charge R3.5747 per kWh excluding VAT*. Durban: eThekwini Municipality.

Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis. University of California, Irvine.

GitHub (2026) *GitHub REST API description (api.github.com OpenAPI document)*. Available at: https://github.com/github/rest-api-description (Accessed: 21 September 2026).

Holm, S. (1979) 'A simple sequentially rejective multiple test procedure', *Scandinavian Journal of Statistics*, 6(2), pp. 65–70.

Huang, L. et al. (2025) 'A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions', *ACM Transactions on Information Systems*, 43(2). arXiv:2311.05232.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A. and Fung, P. (2023) 'Survey of hallucination in natural language generation', *ACM Computing Surveys*, 55(12), Article 248.

JSONPlaceholder (2026) *JSONPlaceholder guide*. Available at: https://jsonplaceholder.typicode.com/guide/ (Accessed: 21 September 2026).

Landis, J.R. and Koch, G.G. (1977) 'The measurement of observer agreement for categorical data', *Biometrics*, 33(1), pp. 159–174.

LibreHardwareMonitor (2026) *LibreHardwareMonitor: open-source hardware monitoring*. Available at: https://github.com/LibreHardwareMonitor/LibreHardwareMonitor (Accessed: 27 September 2026).

Liu, F. et al. (2024) 'Exploring and evaluating hallucinations in LLM-powered code generation'. arXiv:2404.00971.

LM Studio (2026) *LM Studio: local large language model server*. Available at: https://lmstudio.ai (Accessed: 30 September 2026).

McNemar, Q. (1947) 'Note on the sampling error of the difference between correlated proportions or percentages', *Psychometrika*, 12(2), pp. 153–157.

Microsoft (2026) *Pyright: static type checker for Python*, version 1.1.414. Available at: https://github.com/microsoft/pyright (Accessed: 25 September 2026).

Open-Meteo (2026) *Open-Meteo OpenAPI specifications (forecast, air quality, marine)*. Available at: https://github.com/open-meteo/open-meteo/tree/main/openapi (Accessed: 21 September 2026).

OpenAPI Initiative (2021) *OpenAPI Specification, version 3.1.0*. Available at: https://spec.openapis.org/oas/v3.1.0 (Accessed: 21 September 2026).

Patil, S.G., Zhang, T., Wang, X. and Gonzalez, J.E. (2023) 'Gorilla: Large language model connected with massive APIs'. arXiv:2305.15334.

Qwen Team (2026) *Qwen3.5-9B* [GGUF model weights, quantizations Q4_K_M, Q6_K and Q8_0]. Distributed through LM Studio (Accessed: 30 September 2026).

REST Countries (2026) *Countries API v5 documentation*. Available at: https://restcountries.com/docs/countries (Accessed: 21 September 2026).

Spracklen, J. et al. (2025) 'We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs', *34th USENIX Security Symposium*. arXiv:2406.10279.

Stillwell, W.G., Seaver, D.A. and Edwards, W. (1981) 'A comparison of weight approximation techniques in multiattribute utility decision making', *Organizational Behavior and Human Performance*, 28(1), pp. 62–77.

Swagger (2026) *Swagger Petstore: OpenAPI 3 sample*. Available at: https://petstore3.swagger.io/api/v3/openapi.json (Accessed: 21 September 2026).

University of KwaZulu-Natal (2026) *COMP301 (2026) Software Design. Confident Agent: Designing for Trust under Uncertainty*. Group project assignment, version 1.0. School of Mathematics, Statistics and Computer Science.

Vorster, L. (2026a) *Trust by Design: Architectural Patterns for Confident AI Agents*. COMP301 course notes, University of KwaZulu-Natal.

Vorster, L. (2026b) *Trust by Design, not by Declaration*. COMP301 executive presentation, University of KwaZulu-Natal, 1 October.

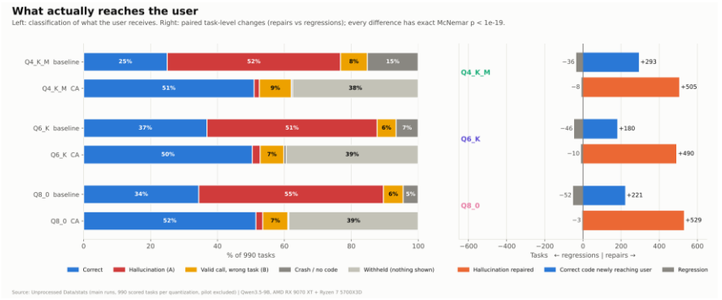
Wilcoxon, F. (1945) 'Individual comparisons by ranking methods', *Biometrics Bulletin*, 1(6), pp. 80–83.

Wilson, E.B. (1927) 'Probable inference, the law of succession, and statistical inference', *Journal of the American Statistical Association*, 22(158), pp. 209–212.

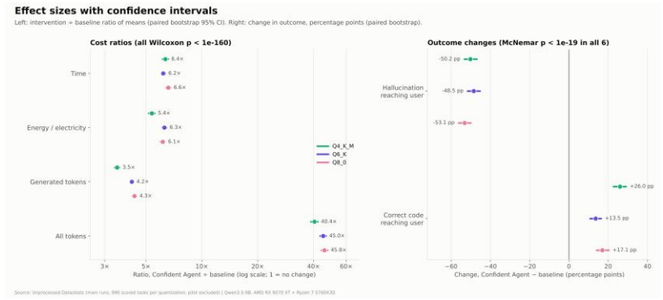
Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B. and Wesslén, A. (2012) *Experimentation in Software Engineering*. Berlin: Springer.

Appendix A: Summary of Results

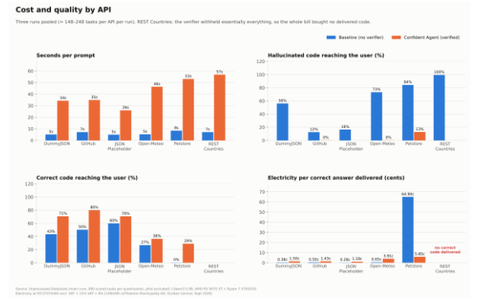
Supplementary charts generated from the unprocessed data (`Findings/Statistical Analysis`). All panels use the 990 scored tasks per quantization (pilot excluded) and price electricity at R4.1109/kWh including VAT. Baseline is blue and the Confident Agent orange throughout.



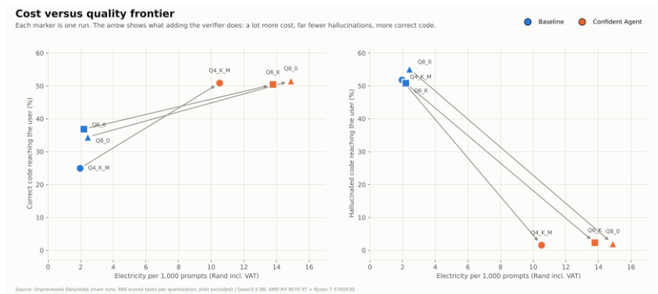
(a) What reaches the user, with paired repairs and regressions



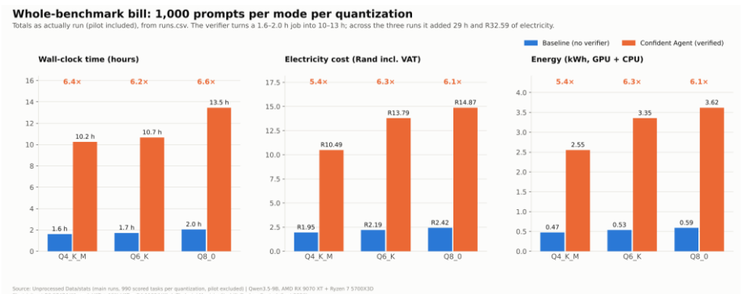
(b) Effect sizes: cost ratios and outcome changes



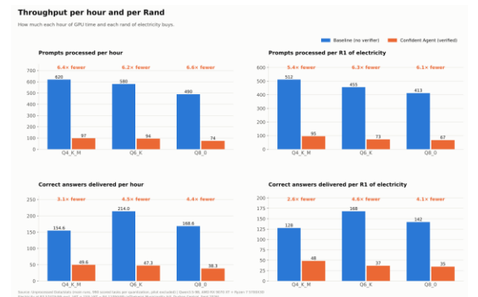
(c) Cost and quality by API



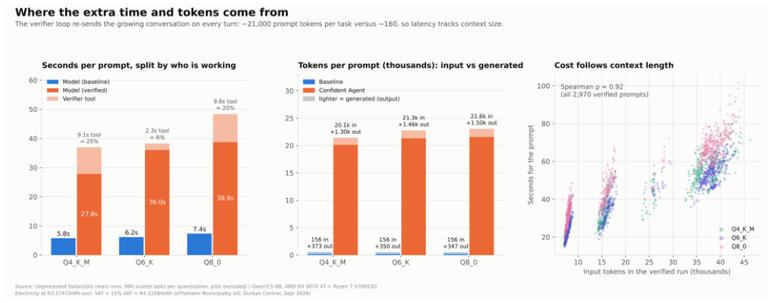
(d) Cost versus quality frontier



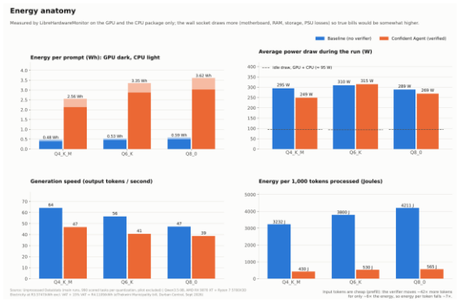
(e) Whole-benchmark time and electricity bill



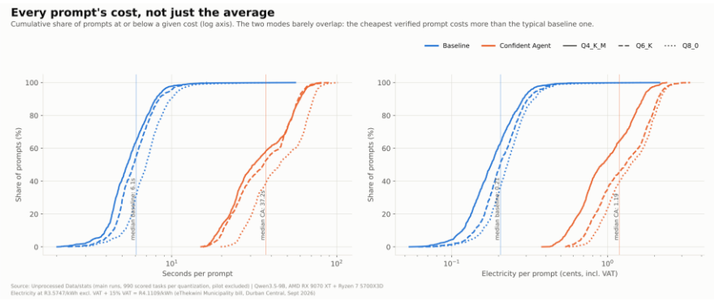
(f) Throughput per hour and per Rand



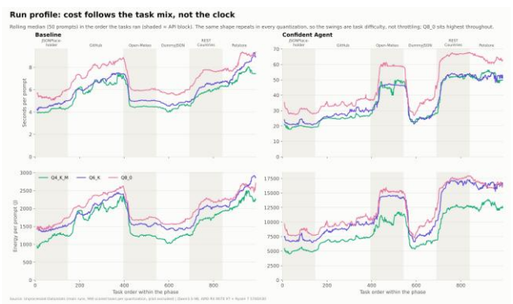
(g) Where the extra time and tokens come from



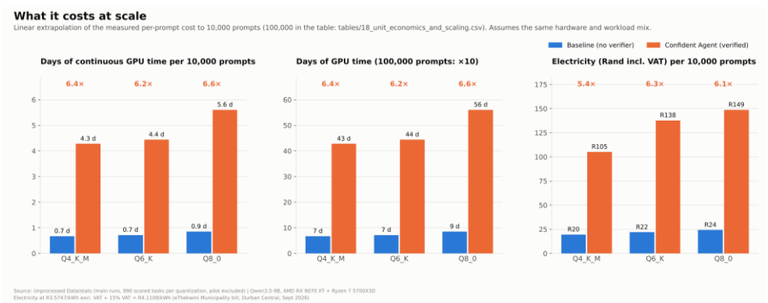
(h) Energy and power anatomy



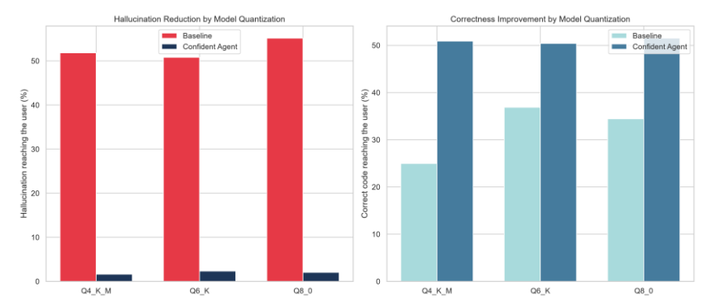
(i) Distribution of per-prompt cost



(j) Run profile: cost follows the task mix



(k) Linear extrapolation to 10,000 and 100,000 prompts



(l) Hallucination and correctness by quantization

Figure 7: Summary of results: supplementary charts (a)–(l).

Appendix B: Benchmark Details

This appendix gives the detail behind Chapter 4.

B.1 Evidence and its provenance

GitHub's specification is large and evolves continuously. Rather than curating it down to the tested operations, the **complete** official document was frozen (13 MB, 819 paths, SHA-256 `4fbdc7d0…`): a hash freezes a large file as firmly as a small one, the full contract cannot omit an operation by curation error, and it lets the evaluator distinguish an invented path (class A) from a real GitHub operation that is simply not the one the task asked for (class B). GitHub prompts pin API version 2026-03-10. Open-Meteo's official specifications (forecast, air quality and marine) were converted from YAML to JSON.

For JSONPlaceholder, DummyJSON and REST Countries no adequate official OpenAPI document exists. Small project-owned specifications were built for the operations the benchmark uses; each cites the documentation pages it was built from, stored with their hashes, and is marked as a researcher-authored subset. REST Countries' authentication was verified against its official documentation, which requires a key sent as `Authorization: Bearer <key>` (or `?api-key=`); the answer key records the Bearer form. The service moved to an authenticated v5 interface on a new host (`api.restcountries.com/countries/v5`), and its older public versions are deprecated. Appendix G lists the hash of every frozen file.

B.2 Task construction and composition

The dataset builder selects documented operations, varies supported optional parameters, request fields and requested response fields, and constructs each record with a checked Python template. Variants that differ only in an identifier or wording do not count as new scenarios. Every record has seven top-level fields (`task_id`, `language`, `natural_language_instruction`, `api_specification`, `ground_truth`, `test_case`, `metadata`). Prompts are neutral: they name the API and describe the operation and its inputs, but never the URL, the method or the parameter names, and never warn the model about hallucination. A typical prompt reads:

```
Write a Python function solve(inputs) for REST Countries v5. Retrieve one page of countries.
Use the caller-supplied credential in inputs["credential"], assumed to have the necessary
permissions. Return the decoded JSON response. Return None for a successful empty response.
Raise an exception for an unsuccessful HTTP response. Assume referenced resources and
accounts
exist. Do not execute the function.
```

The `solve(inputs)` contract makes every candidate callable by the evaluator with the task's test inputs, and the three closing requirements give the behaviour checks. Table 12 gives the composition of the scored tasks.

Table 12**:** Composition of the 990 scored tasks (the 10 pilot tasks are excluded from the analysis).

Dimension	Groups (tasks)	Dimension	Groups (tasks)
Operation	read 494 · create 232 · update 197 · search/filter 37 · delete 30	Specification	official 545 · partial 445
Authentication	none 495 · Bearer 248 · API key 154 · OAuth2 93	Difficulty (heuristic)	easy 267 · medium 720 · hard 3

Difficulty is the dataset's explicit heuristic (number of selected parameters and fields, and whether a credential is needed), not a measured difficulty; with three "hard" tasks it is reported only descriptively.

Tasks were not relabelled to reach target proportions: the generator produces the operations each specification genuinely supports, and the dataset audit (`DATASET_AUDIT.md`) documents the resulting composition. The ten pilot tasks overlap the main dataset; results including them are a robustness check (Section 9.4).

Appendix C: Experimental Setup

This appendix gives the detail behind Chapters 5 and 7.

C.1 Model and run configuration

Table 13**:** The pinned model and run configuration (identical for baseline and intervention; read from the run manifests).

Setting	Value
Model	Qwen3.5-9B (publisher qwen, architecture qwen35, GGUF), loaded in LM Studio as `qwen3.5-9b-q4`, `-q6`, `-q8`
Quantizations	Q4_K_M, Q6_K and Q8_0 (as reported by LM Studio for the loaded model)
Serving software	LM Studio, OpenAI-compatible server on 127.0.0.1:1234; tool use enabled
Context length	30,208 tokens loaded (model maximum 262,144)
Sampling	temperature 0.2; no fixed seed (one sample per task per run); reasoning ("thinking") off
Limits	max 4,096 completion tokens per reply; max 4 model turns per intervention task; 900 s request timeout
Baseline system prompt	"You are an API coding assistant. Write the Python code the user asks for, using the `requests` library. Reply with the complete code in a single fenced python code block."
Intervention system prompt	Confident Agent prompt for toolbox 3.0.0 (SHA-256 `0ebc4dce`…), operator note removed; tool offered: `verify_candidate` only
Prompts sent	rest_api_prompts_v1_DRAFT.jsonl (SHA-256 `c4c68a64`…), identical in both conditions
Software versions	benchmark 2026-09-27.2; toolbox 3.0.0 (decision policy 2026-09-25.1); Pyright 1.1.414; harness Python 3.13.15; Node.js 24.15.0
Machine	Windows 11; AMD Ryzen 7 5700X3D (8 cores); 32 GB RAM; AMD Radeon RX 9070 XT; Durban Central
Run dates (SAST)	Q6_K 30 Sep–1 Oct; Q8_0 1–2 Oct; Q4_K_M 2–3 Oct 2026

C.2 Procedure details

The harness resolves the code that reaches the user in a fixed order: the code the tool delivered on the last verified candidate or, if the model's reply contains code the tool did not approve, that code, recorded as unapproved (this never occurred). Every reply, tool call, verified candidate and verification result is stored, together with a run manifest recording the settings, LM Studio's model information, the machine and the power sensors, and tasks are resumable after an interruption.

C.3 Measurements

Table 14**:** What was measured for every task and condition.

Measure	Definition
Primary outcome	Class A code reaches the user (1/0), as classified by the independent evaluator
Secondary outcomes	class A or B; any error (A, B or runtime failure); correct code reaches the user; incorrect acceptance and rejection; abstention
Verifier behaviour	decision, EWCS, layer states, corrections, protocol deviations, verification calls, tool failures

Measure	Definition
Task behaviour	returns the expected value; returns None for an empty 204 response; raises on a 500 error
Time	wall-clock seconds per task, split into model time (LM Studio) and tool time (verification)
Tokens	prompt and completion tokens summed over the task's requests, as reported by LM Studio
Energy	GPU package and CPU package power from LibreHardwareMonitor, sampled every 0.5 s and integrated over the task; idle power measured for 10 s before each run

AMD GPUs offer no command-line power reading on Windows, so the harness reads LibreHardwareMonitor's local web server (LibreHardwareMonitor, 2026) for the "GPU Package" sensor of the RX 9070 XT and the CPU "Package" sensor. The measured energy excludes memory, storage, motherboard and power-supply losses, so the true wall-socket energy is higher. Energy is converted to Rand at the eThekwini Municipality tariff for the test site in Durban Central for September 2026, **R3.5747 per kWh excluding VAT**, plus 15% VAT, i.e. **R4.1109 per kWh** (eThekwini Municipality, 2026):

$$Cost\ (R)\ =\ \frac{E\ (J)}{3\ 600\ 000\ J/kWh}\ \times\ R3.5747/kWh\ \times\ 1.15\ =\ E\ \times\ 1.1419\ \times\ 10^{-6}\ R/J$$

C.4 The independent evaluator

The evaluator runs each candidate in a separate `python -I -S` process. Its stand-in for `requests` has the real library's names and signatures but sends nothing; sockets, subprocesses and file writes are blocked, and each run has a 10-second limit. It returns a response built from the operation's documented response schema, an empty 204 response or a 500 error. From each run it records the exact requests (method, URL, query as `requests` would encode it, JSON or form body, headers, authentication), the response fields read and whether each is documented, any `requests` function or keyword that does not exist, and the return value or exception. The requests are then classified against the task's pinned specification and expected request with fixed rules (Appendix H).

C.5 Manual review procedure

Each open question (for example: does DummyJSON support a `sort` query parameter? does REST Countries v5 have a `/v5/name/{name}` path?) was checked once against the provider's documentation and, where the documentation was silent, against the live public API with harmless read requests (for example, `?sort=title` returns exactly the unsorted list, while the documented `?sortBy=title` sorts it). Each answer was recorded as an explicit rule, so that every candidate asking the same question received the same verdict, and the evaluation plan's rules were then applied unchanged: a name the provider does not have is class A; a name it has is judged as for an official specification. The three disagreements between the two reviews were JSONPlaceholder candidates that read a resource from a list endpoint (`/comments?id=…` instead of `/comments/{id}`); the group settled them as correct because they return the requested resource. All 396 decisions, with their reasons, are in the submission (`reviews.csv`, `review_rules.py`).

C.6 Changes to the evaluation plan

Table 15**:** Changes to the evaluation plan after it was fixed on 27 September 2026.

Date	Change	Reason and effect
27 Sep	Undeclared request-body fields become unverifiable in the verifier	Found by inspecting the first benchmark run (8 hallucinated fields delivered); disclosed; applies to all reported runs
30 Sep	Holm family limited to the three main runs	Decided before those runs were scored
6 Oct	Findings use the final series of runs (30 Sep–3 Oct); an earlier series is used only as a repeat-run check	Machine throughput halved during the earlier series (GPU power 240 W to 193 W), invalidating its cost figures; its accuracy agrees within about 2 points (Appendix E.10)
6 Oct	Manual review made with AI assistance rather than by two independent members	Time constraints; documented rules and reasons for all 396 decisions
7 Oct	Second, human review of the 46 decisive candidates; three relaxed to correct	$\kappa = 0.88$; changes correct-code figures by one task (Q6_K) and two (Q8_0)
16 Oct	Answer-key audit made by one reviewer rather than two; evaluator audit drawn from the Q6_K run only; both completed after the main analysis	Time constraints; the audits validate the instruments and change no reported outcome; 18 evaluator-audit disagreements await adjudication

C.7 Statistical methods

Table 16**:** Statistical methods (evaluation plan §6).

Question	Method
Does the intervention change the primary outcome?	Exact two-sided McNemar test on the paired binary outcome within each run (McNemar, 1947); $\alpha = 0.05$
How large is the change?	Difference in proportions with 95% percentile bootstrap intervals (10,000 resamples) over tasks and, to allow for related prompts, over the 141 operation families (Efron and Tibshirani, 1993)
Three runs, one hypothesis	Holm's step-down correction across the three runs (Holm, 1979); the runs repeat the same tasks and are not independent replications
Per-group rates	Wilson 95% intervals (Wilson, 1927); subgroup tests are descriptive
Cost	Wilcoxon signed-rank test on paired per-task costs (Wilcoxon, 1945) with rank-biserial effect size; Kruskal–Wallis across quantizations
Confidence metric	Area under the ROC curve, correct versus class A final candidates; share correct per EWCS band
Agreement	Cohen's κ for the manual review, the evaluator audit (each reviewer against the evaluator and against each other), between repeated runs and between quantization runs (Cohen, 1960)
Robustness	Pilot tasks included; undetermined counted as A and as correct; hallucinations other than a wrong host; the five planned weighting schemes

C.8 Summary of threats to validity

Table 17**:** Summary of the main threats to validity and their mitigation.

Threat	Risk	Mitigation
Curated partial specifications	Ground-truth errors	Labelled; undetermined cases reviewed against documentation and live APIs; κ = 0.88
Answer-key errors	Systematic errors in the expected requests	Self-check on 1,000 references; 100-task human audit, all passed; review of open cases
Evaluator built by the same team	Correlated blind spots	Separate code, method and data; sandboxed execution; self-check; two-reviewer audit of 102 decisions
Overfitting to the benchmark	Design tuned to the test set	Pilot tasks excluded; parameters fixed in advance; sensitivity analysis; rule change disclosed
Runtime and sampling drift	Unequal conditions between runs	Pinned model; repeat runs reproduce within about 2 points; aggregate tests only
Weights and thresholds chosen by the group	Result depends on arbitrary numbers	Five pre-planned schemes: no change to withheld or delivered outcomes
Combined intervention	Effect attributed to verification is partly prompting	Stage decomposition (Section 8.3); stated as not an ablation
Related observations	Overstated precision or independence	Tests within each run; cluster bootstrap over families; runs treated as robustness
One model, six APIs, Python	Limited generality	Stated scope; design is API-agnostic but evidence-dependent

Appendix D: Confident Agent Implementation

This appendix gives the implementation detail behind Chapter 6.

D.1 The Trust-by-Design patterns

Table 18**:** How the Confident Agent realises the Trust-by-Design patterns (Vorster, 2026a).

Pattern	Realisation in the Confident Agent
Independent Verification	The model proposes code; a deterministic tool (claim extractor, Pyright, OpenAPI matcher) judges it. The verifier never asks the model whether it is right.
Domain-Specific Validation	Evidence is the API's own contract: OpenAPI paths, parameters, schemas and security schemes, plus the Python package registry and type checker.
Evidence Fusion	Six layer verdicts, each with an evidence tier, are combined by the EWCS; missing and contradictory evidence are kept apart (unverifiable versus verified incorrect).
Confidence Gating	The decision policy releases code only if no layer is contradicted, no critical check is unresolved, and the EWCS clears a threshold; it can deliver, flag, request a correction, or abstain.
Deferred Commitment	A contradicted candidate is not released; the agent may rewrite it once and must have the new code verified. An abstention without a contradiction is final.
Provenance Preservation	Every verification records the candidate's SHA-256, the evidence file and its hash, per-layer evidence details and the decision; the report renderer refuses code whose hash differs from the verified candidate.

D.2 Where the tool attaches, and why

Table 19**:** Options for attaching verification to the model.

Option	Advantages	Disadvantages
A. External post-hoc controller (CLI): generate, then verify and correct outside the model	Full enforcement; deterministic; the model cannot skip a step	Not how developers use assistants; corrections need a second prompt the model did not plan for
B. In-loop tool via function calling (chosen)	The deployable artefact: a model with a prompt and a tool, as in LM Studio; the model sees the evidence and can correct itself	The model controls the protocol and may deviate; deviations must be measured
C. MCP server	Portable across MCP clients	Adds a server process; LM Studio's tools-provider SDK gives the same function-calling interface natively
D. Multi-agent pipeline (chief agent with specialist LLM agents)	Specialist agents per check	Each agent is another model that can hallucinate; deterministic checks are cheaper and stronger evidence

Option B was chosen because it evaluates the package a developer would actually run. Option A is retained as the toolbox's external controller, used for the checks of the verifier (Section 6.6). Option C would be a packaging change rather than a design change. Option D was rejected on evidence grounds: each specialist agent would be another model that can hallucinate, whereas a deterministic checker cannot (Vorster, 2026a).

D.3 Claim extraction

The extractor follows variables, f-strings, concatenation, dictionaries built key by key, `if`/`try` branches and helper functions that return `response.json()`. Each claim records its type (import, method call, endpoint, input field, output field), its line and its provenance. When the extractor meets a construct it cannot follow (a `requests.Session`, another HTTP client, a class, async code), it reports an issue that names the layers it affects, and only those layers are downgraded to unverifiable, so that one unsupported construct cannot make every layer unverifiable.

D.4 The six layers in detail

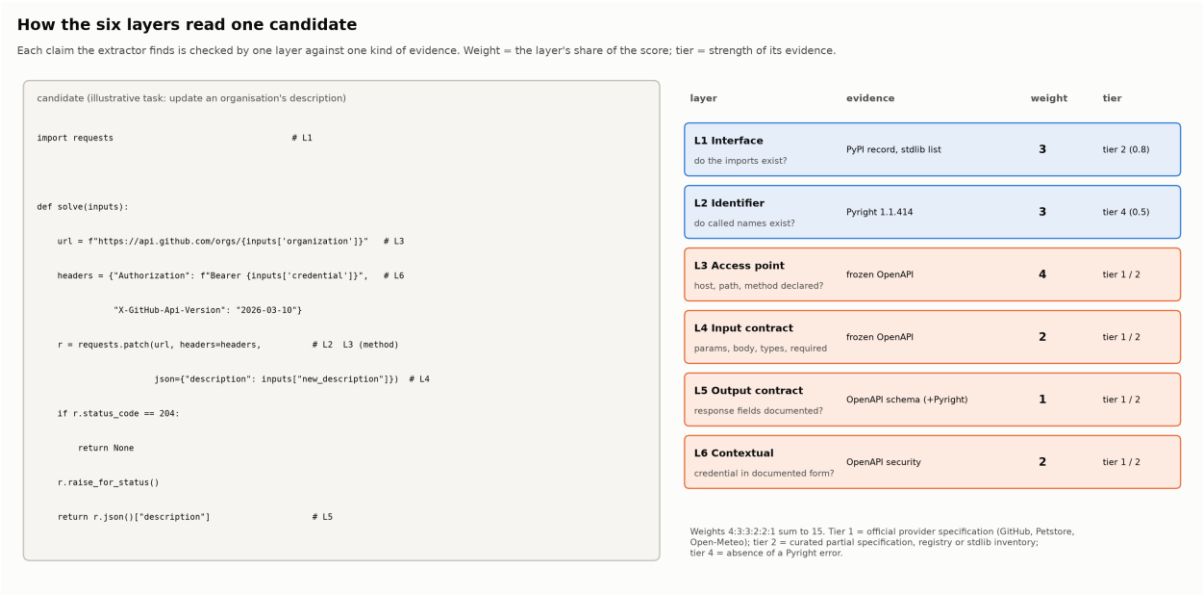


Figure 8: How the six layers read one candidate: each claim is checked by one layer against one kind of evidence.

- **L1 Interface.** Every imported module must exist: standard-library modules against the pinned interpreter's module list, third-party packages against a PyPI record (frozen for `requests` in the experiment so that a network timeout cannot change a verdict).
- **L2 Identifier.** Pyright 1.1.414 in standard mode type-checks the exact candidate. An undefined name, attribute or call is a contradiction; argument-type errors count only on API lines. Absence of an error is weak evidence (tier 4).
- **L3 Access point.** The host and base path must match a server of the frozen specification, and the path template and HTTP method must match a declared operation. A mismatch is a contradiction whose evidence names the closest declared operations, which the agent can use to correct. An unknown host has no evidence and makes the layer unverifiable.
- **L4 Input contract.** Required parameters, their locations (path, query, header, body), types and request-body properties are checked against the matched operation. Runtime values are not guessed; an undeclared query parameter or body field is unverifiable unless the schema explicitly allows or forbids extra properties.
- **L5 Output contract.** Every response field the code reads is followed through the success-response schema. A field the schema documents verifies; a field it does not document cannot be verified, and contradicts a schema that excludes it.

- **L6 Contextual.** The operation's security requirements are checked against the actual request: a missing credential or a wrong scheme is a contradiction; OAuth2 scopes, which are granted to a token rather than expressed in code, are reported as a note.

A layer aggregates its claims by taking the most severe verdict, so one contradicted parameter contradicts the layer.

D.5 Frozen evidence, selected by host

Specifications could in principle be discovered at run time, by probing well-known paths or searching the web. The toolbox does not do so, for two reasons: a live specification can change between the baseline and agent runs, which would destroy the comparison, and four of the six benchmark APIs publish no discoverable specification, so a dynamic resolver would measure discovery rather than verification. The toolbox instead bundles the same frozen snapshots as the dataset, each with its provenance and hash, and a registry maps each host to its snapshot.

D.6 The EWCS: signed score, weights, tiers and ceilings

The EWCS lies in [0, 1]. It is 1 only when every applicable layer is verified by authoritative (tier 1) evidence, and it falls both when evidence is weaker and when layers are unverifiable or contradicted. Not-applicable layers leave the denominator, so a candidate is not penalised for having no response fields to read. A diagnostic **signed score** S keeps contradictions visible by using σ_l = +1 (verified correct), −1 (verified incorrect) or 0 (otherwise), so that S lies in [−1, 1]:

$$S = \frac{\sum_{l \in A} w_l \cdot t_l \cdot \sigma_l}{\sum_{l \in A} w_l}$$

The weights are a design hypothesis, fixed before the main runs and then tested (Section 8.5). They come from a two-step procedure that is standard in multi-criteria decision analysis when only the *order* of importance can be justified: rank the criteria, then convert ranks to weights with the **rank-sum rule** (Stillwell, Seaver and Edwards, 1981; Barron and Barrett, 1996):

$$w_k \propto n - r_k + 1, \; n = 4 \; severity \; ranks \; \Rightarrow \; 4 : 3 : 2 : 1$$

We ranked the layers by the consequence of a hallucination in that layer for the user, judged on two criteria: whether the call can still do anything useful, and whether the failure is silent or loud (Table 20).

Table 20**:** Severity ranking of the layers and the rank-sum weights (total 15).

Rank	Layer	Weight	Why
1	L3 Access point	4	A wrong host, path or method sends the request to a resource that does not exist or to the wrong one; nothing downstream can be right. It is the dominant hallucination kind in the baseline, and layers 4–6 are only defined relative to the matched operation.
2	L1 Interface, L2 Identifier	3	A non-existent module or function makes the program fail outright. Fatal, but loud: the developer sees an error at once.
3	L4 Input contract, L6 Contextual	2	A wrong parameter or credential usually produces an HTTP error or a partial result. Serious but situational, and frequently loud (4xx responses).
4	L5 Output contract	1	Reading an undocumented field is a downstream data bug; the request itself succeeds.

The tier factor discounts a verdict by the strength of the evidence behind it, on the familiar principle of evidence hierarchies that primary, authoritative sources outrank derived ones and that an absence of contrary evidence is weaker than positive evidence (Table 21).

Table 21**:** Evidence tiers and factors.

Tier	Factor	Sources	Rationale
1	1.0	Provider-published OpenAPI (GitHub, Petstore, Open-Meteo)	The provider's own contract: the strongest evidence available without calling the service.
2	0.8	Curated partial specifications; PyPI record; interpreter module list	One step removed from the authority (transcribed from documentation, or registry metadata); a 20% discount reflects transcription and coverage risk.
3	0.9	Runtime execution (reserved, not used)	Observed behaviour would be stronger than metadata but weaker than the contract it may not exercise. The agent never executes code, so tier 3 never occurs.
4	0.5	Pyright found no error	Absence of a type error is "no evidence against", not evidence for: half credit, the point of indifference.
none	0	No evidence	No evidence earns no credit.

Worked example. For the GitHub candidate of Figure 8, assume all claims verify: L1 by the PyPI record (tier 2: 3 × 0.8 = 2.4), L2 by a clean Pyright run (tier 4: 3 × 0.5 = 1.5), L3, L4 and L5 by GitHub's official specification (tier 1: 4 + 2 + 1 = 7). GitHub's OpenAPI document does not model authentication, so L6 is not applicable and the denominator is 3 + 3 + 4 + 2 + 1 = 13. Hence EWCS = 10.9 / 13 = **0.838**, and the decision is *deliver with flag*. If the same code had called the invented path `/organizations/{org}`, L3 would be verified incorrect, L4 and L5 could not be matched to an operation (unverifiable), and EWCS = 3.9 / 13 = **0.30**; the score is irrelevant, however, because the contradiction triggers the safety constraint. This is why hallucinated candidates cluster at an EWCS of about 0.3 in Figure 15.

Because the weights and tiers are fixed, the highest EWCS each kind of evidence can reach is known in advance (Table 22). The *deliver* threshold of 0.85 was set at the point that requires authoritative evidence for every API layer *including* authentication: only fully modelled official specifications can reach it. Every candidate checked against a curated specification can at most be *delivered with a flag*, which is the honest outcome for researcher-authored evidence. Since a candidate that passes the uncertainty gate has every applicable layer verified, its EWCS is at least that of the weakest row, 0.65, which is why the 0.60 threshold never binds.

Table 22**:** Maximum attainable EWCS when every applicable layer verifies (L1 tier 2, L2 tier 4).

Evidence for layers 3–6	Applicable weight	Max EWCS	Best decision
Official, all of L3–L6 applicable (e.g. Petstore with security)	15	0.860	deliver
Official, no response fields read (L5 n/a)	14	0.850	deliver
Official, authentication not modelled (GitHub, Open-Meteo)	13	0.838	deliver with flag
Official, L5 and L6 n/a	12	0.825	deliver with flag

Evidence for layers 3–6	Applicable weight	Max EWCS	Best decision
Curated partial specification (JSONPlaceholder, DummyJSON, REST Countries)	12–15	0.725–0.740	deliver with flag
No API call (only L1 and L2 applicable)	6	0.650	deliver with flag

D.7 Static and dynamic structure

Figure 9 shows the main types of the verification core and Figure 10 the interaction for one benchmark task.

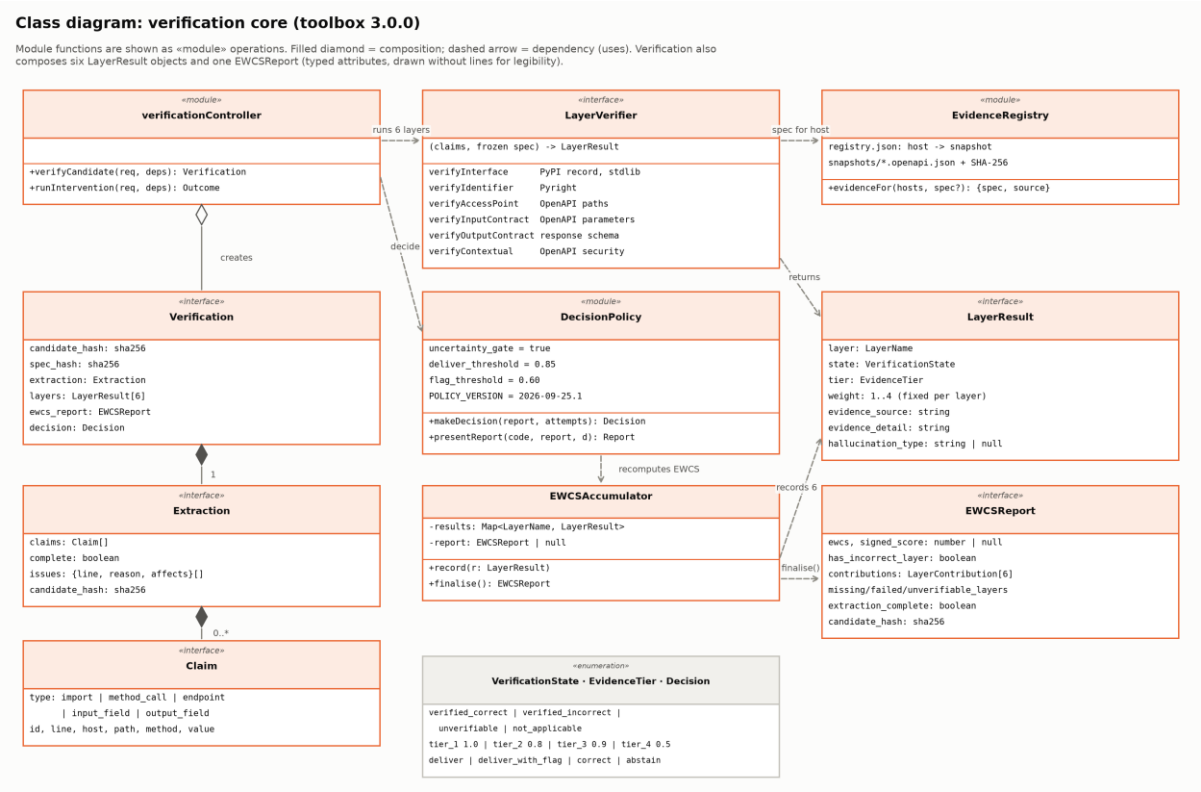


Figure 9**:** Class diagram (UML) of the verification core in toolbox 3.0.0.

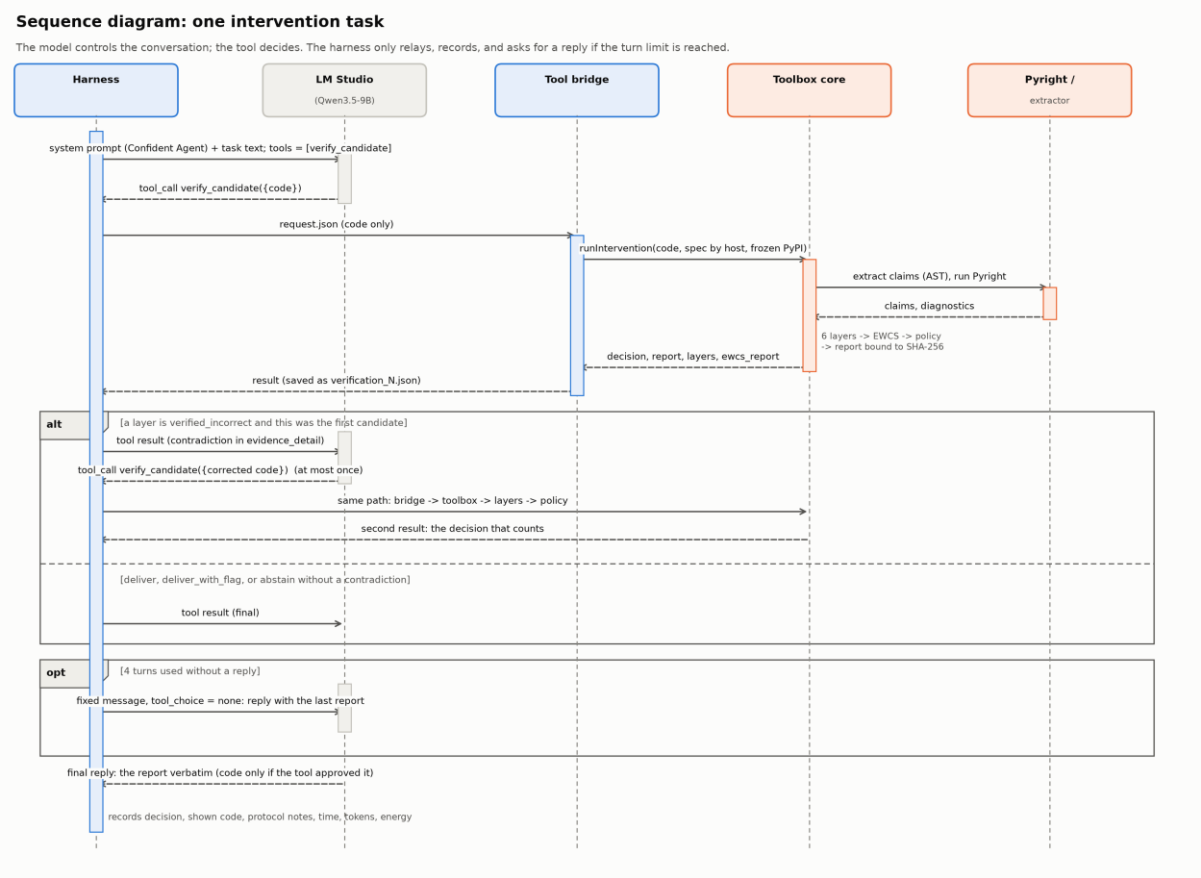


Figure 10**:** Sequence diagram (UML) of one intervention task, including the single permitted correction and the reply request at the turn limit.

Appendix E: Supplementary Results

This appendix gives the supporting analyses behind Chapters 8 and 9. All figures use the 990 scored tasks per run after the manual review; pooled figures add up the three runs over the same tasks and are descriptive.

E.1 Scorecard

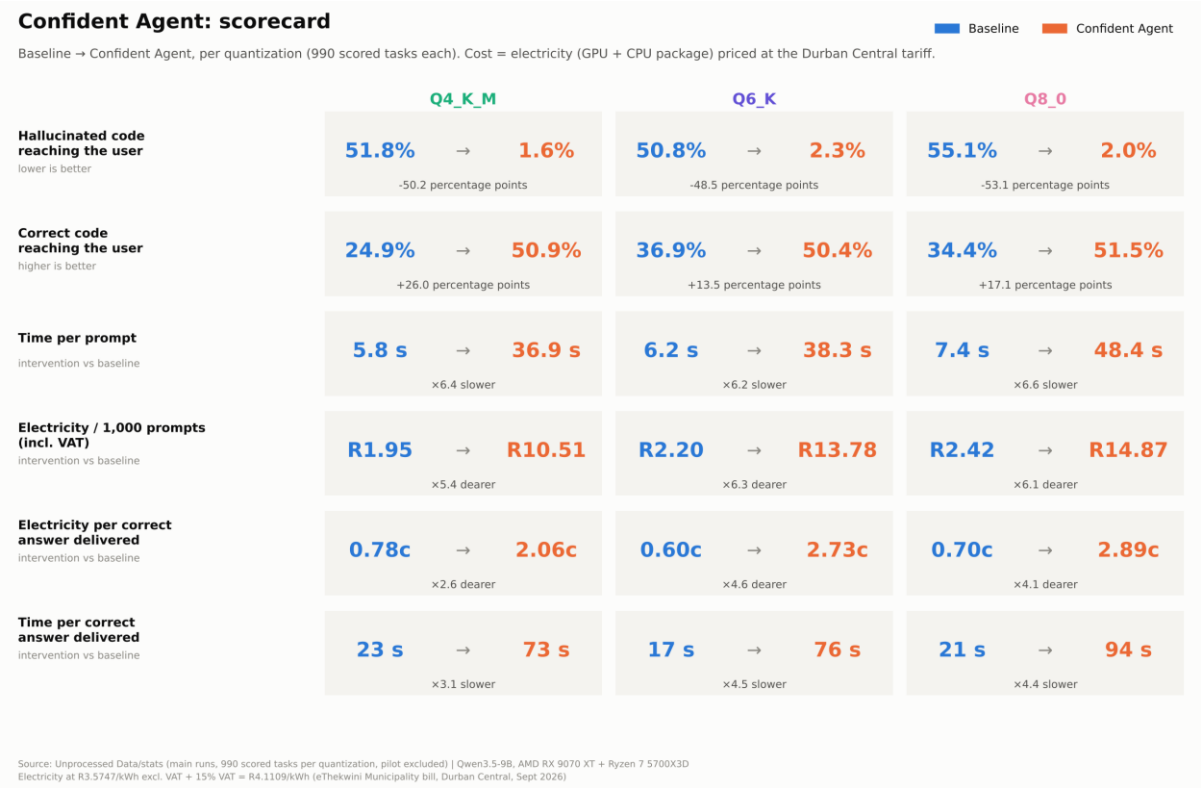


Figure 11**:** Scorecard: baseline versus Confident Agent for each quantization (990 tasks each). Electricity priced at R4.1109/kWh including VAT.

Pooled over the three runs, hallucinated code reached the user in 1,562 of 2,970 task-runs (52.6%) without the agent and in 59 (2.0%) with it, and correct code in 953 (32.1%) and 1,513 (50.9%). The agent's final accuracy is almost the same at every quantization (50.4–51.5% correct) although the baselines differ: the lower-precision model gains most (+26.0 points of correct code at Q4_K_M) because the agent repairs the coding errors in which Q4_K_M's baseline was weakest.

E.2 Results by subgroup

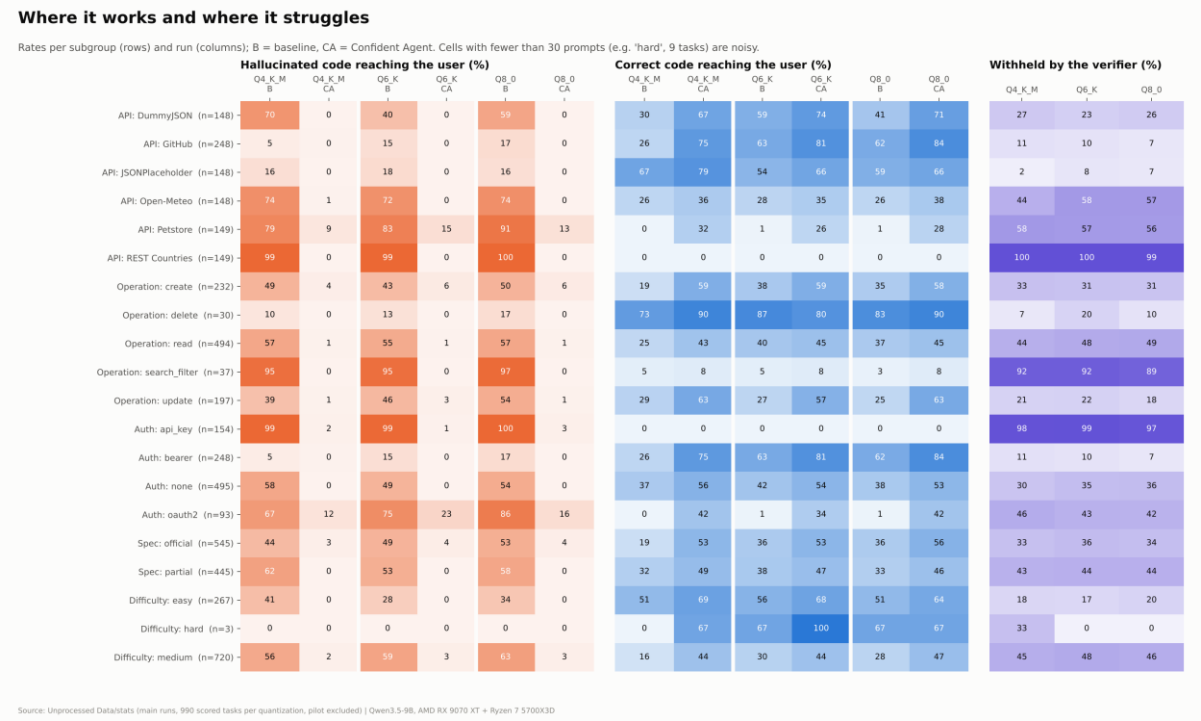


Figure 12**:** Rates per subgroup and run: hallucinated code reaching the user, correct code reaching the user, and code withheld by the verifier (B = baseline, CA = Confident Agent).

APIs the model knows well improve the most: GitHub (Bearer authentication) rises from 26–63% correct to 75–84%, and DummyJSON from 30–59% to 67–74%, with no hallucination reaching the user in any run. Petstore carries almost all the residual hallucinations (9–15% of Petstore tasks), and REST Countries is entirely withheld (Section 8.4).

E.3 Kinds of hallucination

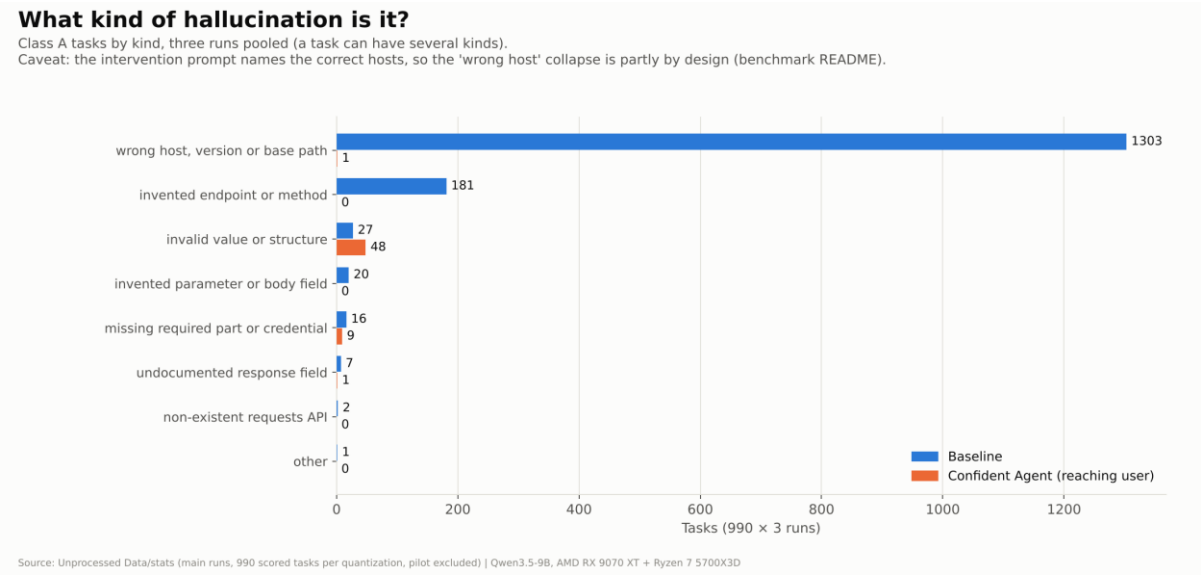


Figure 13**:** Kinds of API hallucination in the code reaching the user (three runs pooled; a task can have several kinds).

Wrong hosts, versions and base paths make up most baseline hallucinations, and they almost vanish in the delivered code. One kind grows: *invalid value or structure*, from 27 to 48 pooled tasks. In 48 of the 59 residual hallucinations a Petstore request body places a value from `inputs` where the schema expects a string (for example `category.name` set to the object `{"id": 1}`), and in 9 more a required `api_key` credential is missing. The cause is a deliberate design decision: the verifier treats values that only exist at run time as opaque, so it cannot see *which* value lands in a field, whereas the evaluator binds the task's test inputs and can. Binding representative inputs during verification, which the extractor already supports, is the most direct improvement to the design.

E.4 The verifier as a hallucination detector

Treating the verifier as a classifier of the agent's final candidate (positive = class A, predicted positive = withheld) gives the confusion matrices of Figure 14. Recall is **94–96%**: it lets very few hallucinations through. Precision is **86–92%**: most of what it withholds really is hallucinated. Its false alarms (20–28 correct candidates withheld per run, plus some wrong-task and crashing candidates) are the incorrect rejections the brief asks us to count. When class B errors are also treated as positives, recall drops to 76–80%: the verifier checks the API's contract, not whether the call is the one the task wanted.

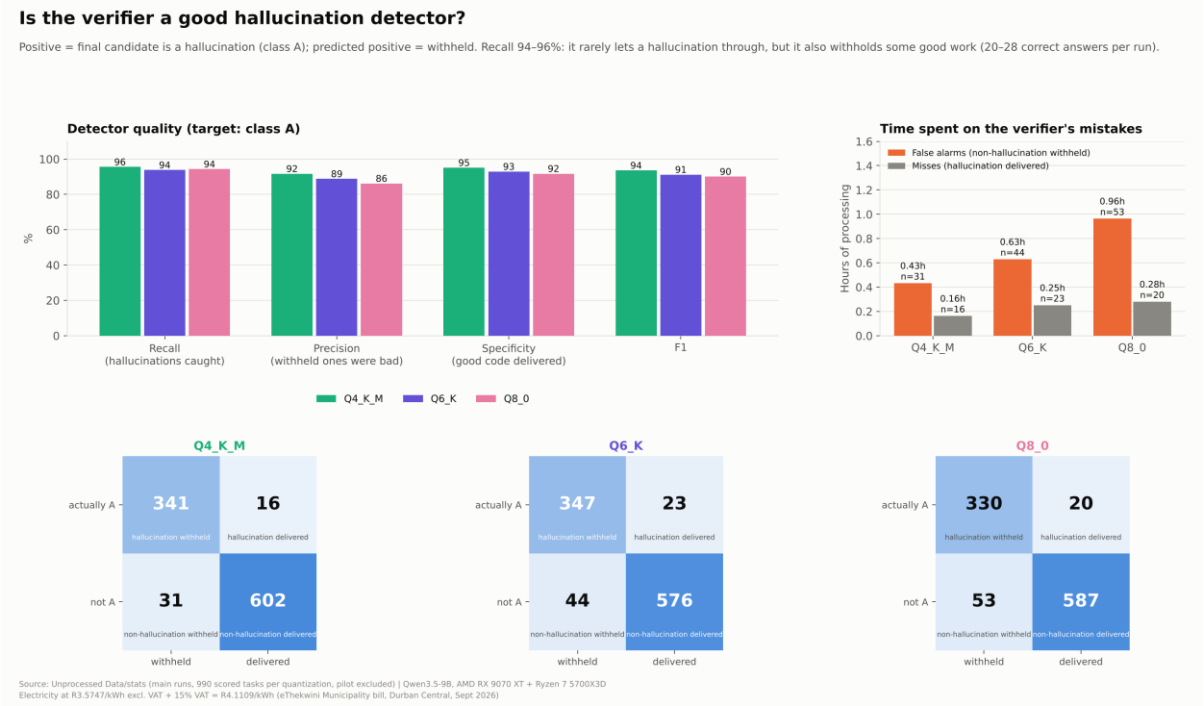


Figure 14**:** The verifier as a detector of hallucinated final candidates: quality measures, confusion matrices, and the processing time spent on its errors.

E.5 Discrimination and calibration of the EWCS

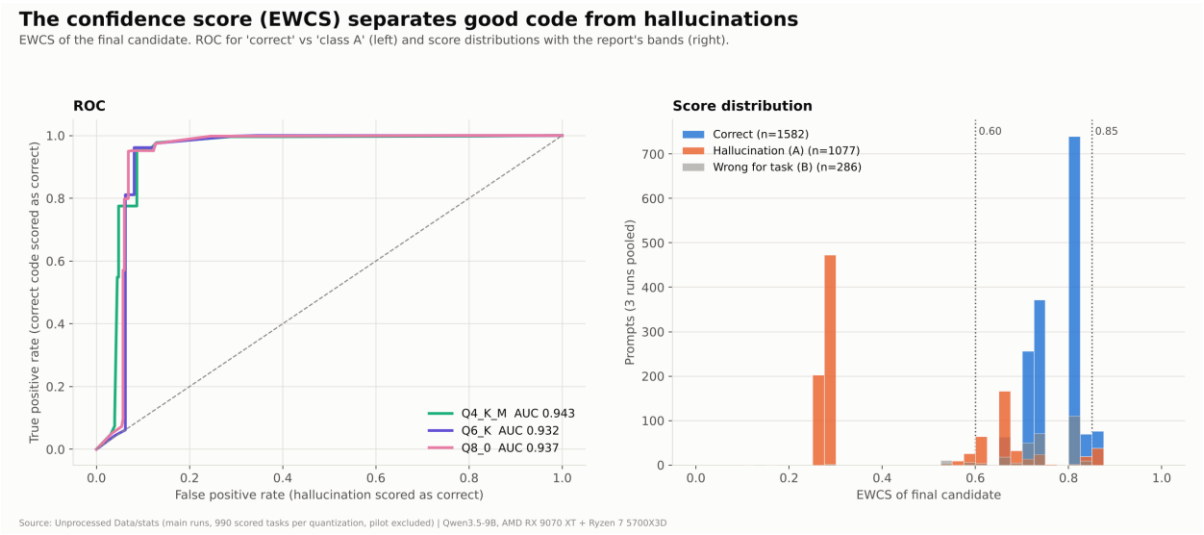


Figure 15**:** EWCS of the final candidate: ROC curve for correct versus class A, and score distributions with the decision thresholds (three runs pooled).

The EWCS separates correct from hallucinated final candidates with an area under the ROC curve of 0.932–0.943 (Figure 15). Hallucinated candidates cluster near 0.3, the value left when the access point is contradicted and the API layers that depend on it cannot be matched (Appendix C.6).

Table 23**:** Share of decided final candidates (correct, A or B) that were correct, by EWCS band (three runs pooled).

EWCS band	Candidates	Correct	Share correct
≥ 0.85 (deliver)	166	110	66.3%
0.60–0.85 (deliver with flag)	2,041	1,469	72.0%
< 0.60 (abstain)	738	3	0.4%

Candidates scoring 0.85 or more were correct only 66% of the time, *less* often than the flagged band, because the high band is reachable only for Petstore, the API with most residual errors and the most task-fidelity (class B) mistakes. The score measures how much of the claim the evidence covers, not how likely the code is to do what the user asked. The low band, by contrast, is almost pure error: 3 of 738 candidates below 0.60 were correct. Correct candidates cluster at the evidence ceilings of Table 22 (0.74 for curated, 0.825–0.838 for official specifications).

E.6 Sensitivity to the weights, thresholds and gate

The weights and thresholds were treated as a design hypothesis: all five schemes were fixed in advance and all are reported. Figure 16 charts Table 9. The re-scoring is a fair counterfactual because the agent's only behavioural trigger, a correction, depends on contradictions and not on the score.

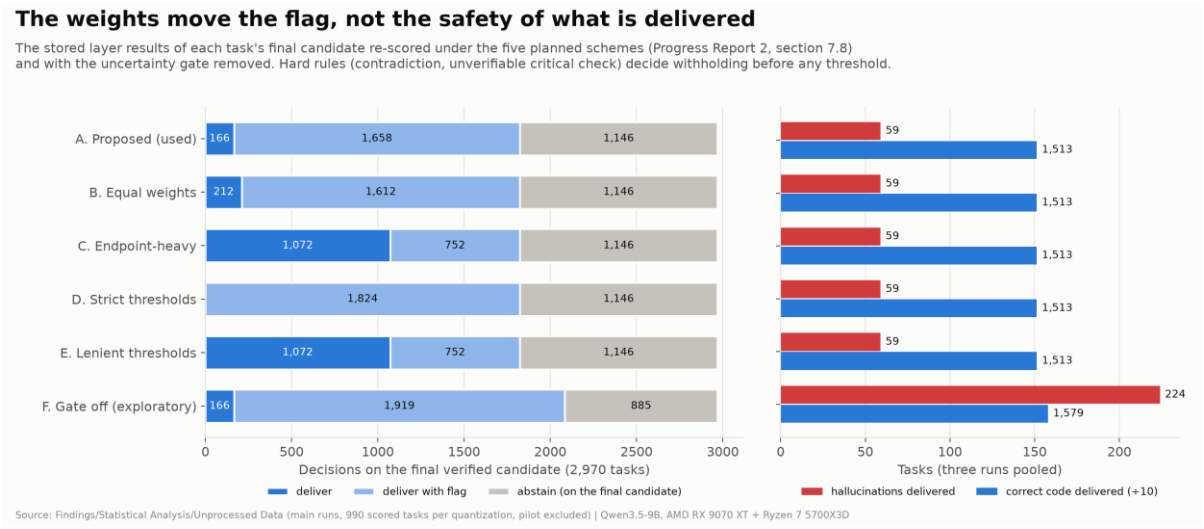


Figure 16: Decisions on the final candidate under the five planned schemes and with the uncertainty gate removed (three runs pooled).

E.7 Correction and agent behaviour

Table 24: Class of the first verified candidate → class of the final candidate, for tasks in which the code changed.

Transition	Q4_K_M	Q6_K	Q8_0
A → correct (hallucination repaired)	121	123	150
A → A (still hallucinated; withheld)	164	165	166
A → B (valid call, wrong task)	36	33	36
Runtime failure → correct	23	2	0
Corrections attempted / permitted	403 / 349	425 / 377	444 / 387
Final candidate correct after a correction	37.7%	34.1%	37.8%

Self-correction works about a third of the time. Figure 17 shows that more correction rounds rarely pay: prompts that needed two or three rounds cost 57–61 s and ended in correct code only 15% and 1% of the time. The model often bent the protocol (Figure 18): 34–39% of tasks reached the four-turn limit and needed the reply request, about 30% re-verified identical code, and 15–19% made more than one correction although the prompt permits one. Only 53–54% of tasks followed the protocol exactly.

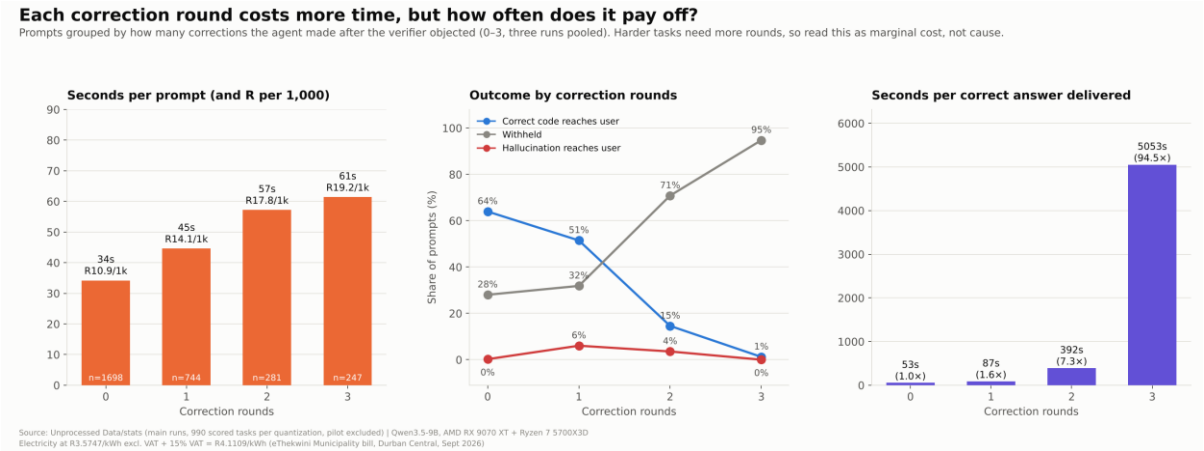


Figure 17: Cost and outcome by the number of correction rounds the agent made (three runs pooled).

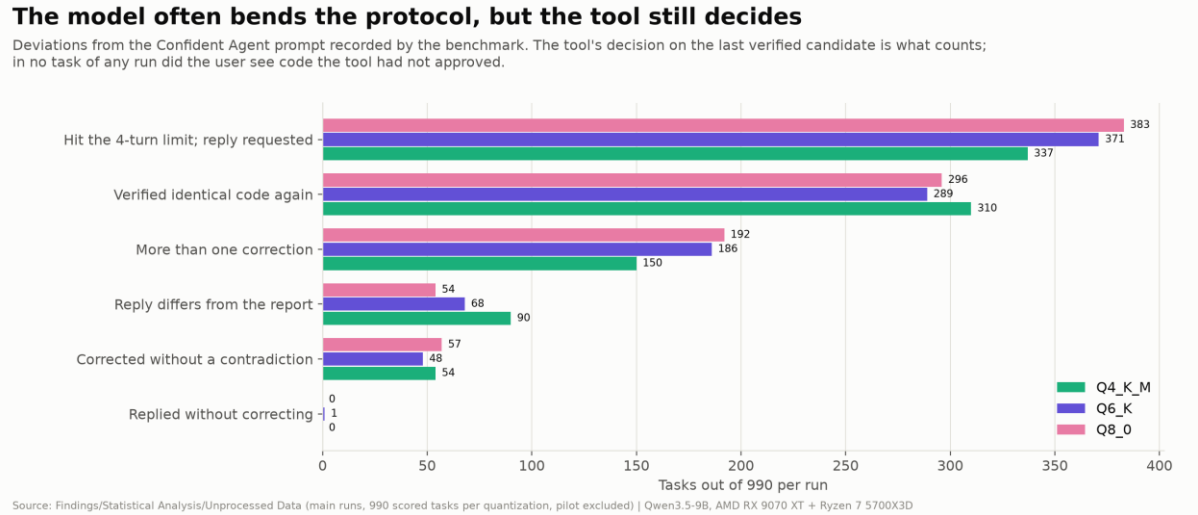


Figure 18**:** Deviations from the Confident Agent prompt recorded by the benchmark, per run.

E.8 Behaviour beyond API correctness

Table 25**:** Task-behaviour checks on code that reached the user (share of programs that make a request).

Requirement	Q4 base	Q4 agent	Q6 base	Q6 agent	Q8 base	Q8 agent
Returns the expected value	80%	83%	86%	82%	86%	84%
Returns None on an empty 204 response	12%	25%	21%	24%	22%	26%
Raises on a 500 error	100%	100%	100%	100%	100%	99%
All three met	8.7%	19.4%	18.5%	22.3%	17.7%	22.5%

Code reaching the user nearly always raises on a server error and usually returns the expected value, but few programs return `None` for an empty 204 response as instructed, in either condition. Among the agent's delivered programs that sent the right request, only 23–27% handled the empty response. Requiring the right request *and* all three behaviours, the agent delivered fully correct code for 10.3%, 10.5% and 11.3% of tasks (Q4_K_M, Q6_K, Q8_0), against 2.1%, 5.8% and 6.5% for the baseline (`stage_analysis.json`).

E.9 The cost of the intervention

Every prompt costs more under the intervention: in all 18 paired Wilcoxon tests (time, energy, total, generated, GPU and CPU energy, for each quantization) $p < 10^{-160}$, the rank-biserial correlation is at least 0.995, and 99.6–100% of individual prompts cost more (Figure 19).

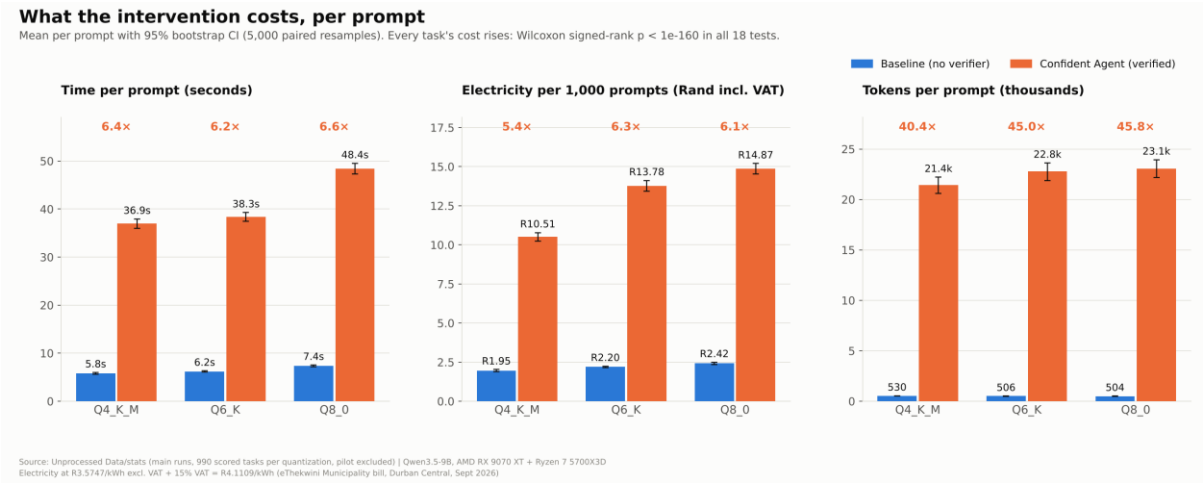


Figure 19**:** Time, electricity (Rand per 1,000 prompts) and tokens per prompt, with 95% bootstrap intervals.

Table 26**:** Cost of the intervention in time, energy and Rand (990 tasks per run; electricity at R4.1109/kWh incl. VAT).

Measure	Q4_K_M	Q6_K	Q8_0
Time per prompt, baseline → agent	5.8 → 36.9 s	6.2 → 38.3 s	7.4 → 48.4 s
Energy per prompt (GPU + CPU)	0.48 → 2.56 Wh	0.53 → 3.35 Wh	0.59 → 3.62 Wh
Electricity per prompt	0.20c → 1.05c	0.22c → 1.38c	0.24c → 1.49c
Tokens per prompt (input + output)	530 → 21,420	506 → 22,771	504 → 23,058
Time per correct answer delivered	23 → 73 s	17 → 76 s	21 → 94 s
Electricity per correct answer delivered	0.78c → 2.06c	0.60c → 2.73c	0.70c → 2.89c
Extra electricity per hallucination prevented	1.70c	2.39c	2.34c
Extra time per hallucination prevented	1.03 min	1.10 min	1.29 min
Whole run, 1,000 prompts: hours	1.6 → 10.2	1.7 → 10.7	2.0 → 13.5
Whole run, 1,000 prompts: Rand	R1.95 → R10.49	R2.19 → R13.79	R2.42 → R14.87

Measured energy covers the GPU and CPU packages only; wall-socket energy is higher. Whole-run figures include the 10 pilot tasks. Across the three runs the verifier added 29.0 hours and R32.59 of electricity (R28.34 before VAT).

At these rates, 10,000 prompts take 0.7–0.9 days on this machine without the verifier and 4.3–5.6 days with it (R20–R24 against R105–R149 of electricity). Figure 20 expresses the cost per good outcome.

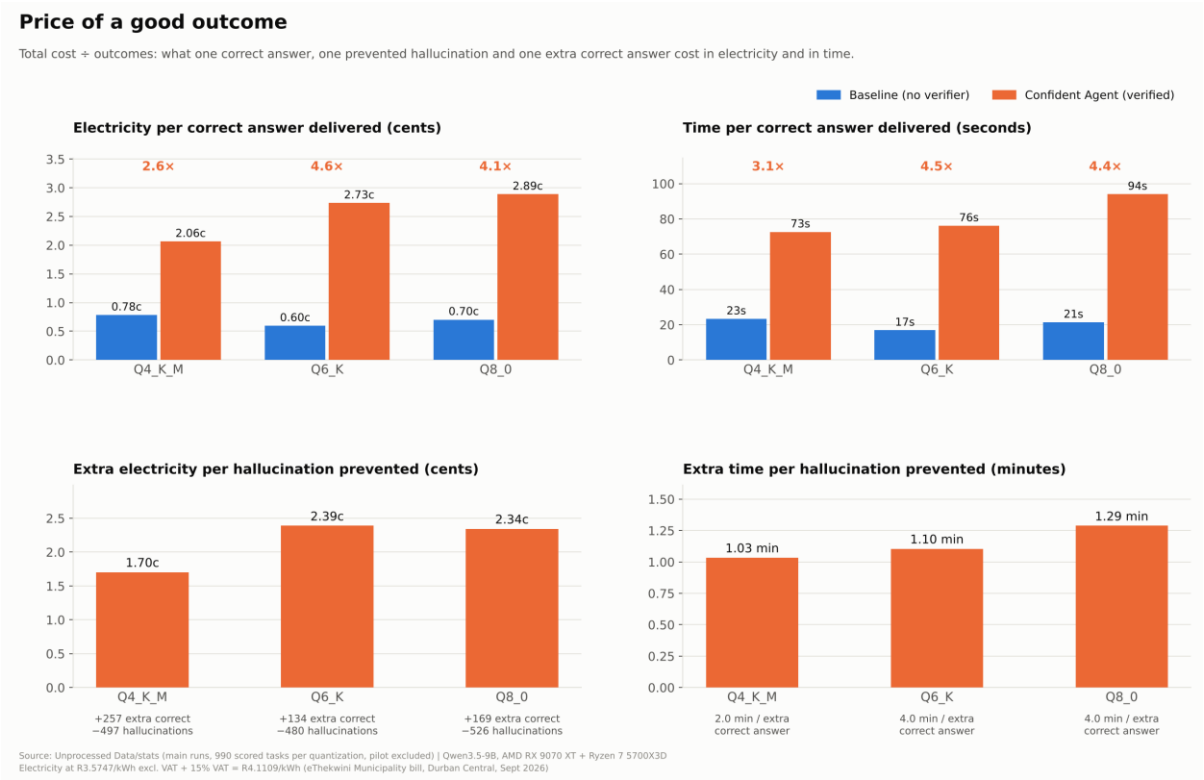


Figure 20**:** What one correct answer, one prevented hallucination and one extra correct answer cost in electricity and time.

Why the cost is so high. Verification takes 2.3–9.6 s per prompt (6–25% of the time). Each turn re-sends the whole growing context, including the six-layer report, so a verified prompt processes about 20,000–21,600 input tokens against 156 for the baseline (about 130 times as many) and generates 3.5–4.3 times as many tokens; input length alone tracks the time per prompt (Spearman ρ = 0.92). Energy rises less than tokens because input tokens are cheap to process. The 34–39% of tasks that hit the turn limit pay for the extra reply round as well.

Where the money goes. About 35–38% of the agent's processing time is spent on tasks that end in a correct delivery, and about half on tasks whose final candidate was a hallucination that was then withheld: 5.0–6.2 hours per run (R5.01–R6.65), money well spent from a safety standpoint but buying no deliverable (Figure 21). False alarms cost 0.4–1.0 hours per run. REST Countries alone consumed 7.1 hours and R7.93 across the three runs and delivered nothing.

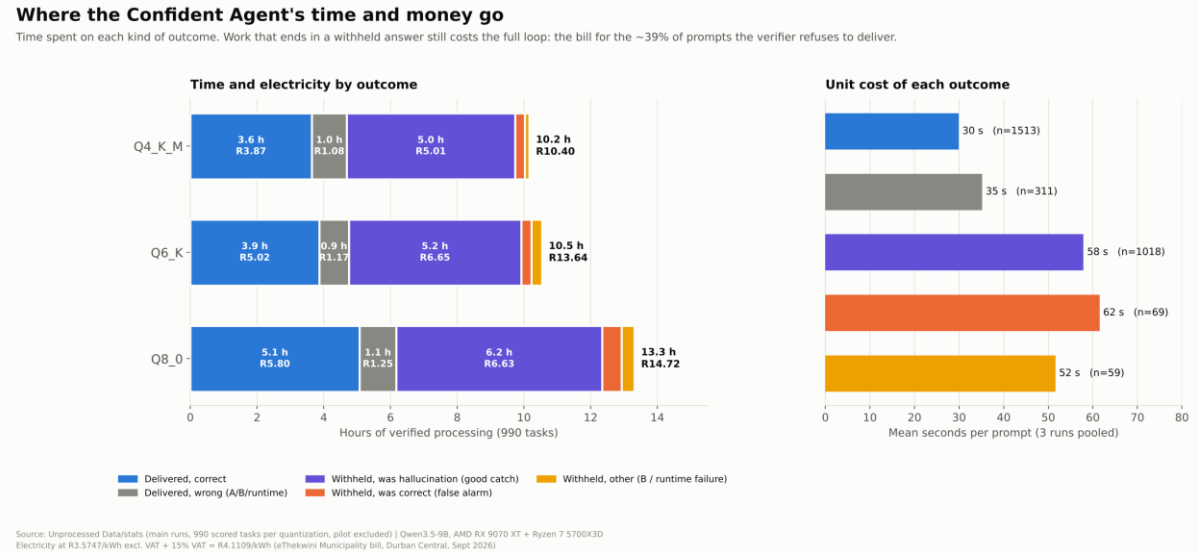


Figure 21**:** Where the Confident Agent's time and electricity go, by outcome.

E.10 Quantization, run-to-run reliability and agreement between runs

With the verifier, accuracy is almost independent of quantization (50.4–51.5% correct, 1.6–2.3% hallucinated), so the cheapest model is the best value: Q4_K_M needs 36.9 s and R10.51 per 1,000 prompts against 48.4 s and R14.87 for Q8_0, and delivers 48 correct answers per Rand of electricity against 35. Cost differences between quantizations are statistically clear (Kruskal–Wallis $p < 10^{-49}$ for time and energy in both conditions), but accuracy differences are descriptive: each quantization ran once, on a different day.

Run-to-run variation can be estimated because Q4_K_M and Q8_0 were each run twice with identical settings (the earlier series was superseded for its cost figures, Appendix C.6). Table 27 compares them. The headline rates agree within about two percentage points, but individual tasks vary: at temperature 0.2 without a fixed seed, 11–14% of tasks changed between delivered and withheld from one run to the next.

Table 27**:** Repeat runs with identical settings: earlier series (superseded) versus final series.

Measure	Q4_K_M	Q6_K	Q8_0
Hallucination reaching user, earlier series	50.8% → 1.6%	51.1% → 2.9%	53.1% → 2.0%
Hallucination reaching user, final series	51.8% → 1.6%	50.8% → 2.3%	55.2% → 2.0%
Correct code reaching user, earlier series	23.9% → 50.5%	37.1% → 48.8%	36.5% → 50.3%
Correct code reaching user, final series	24.9% → 50.9%	36.9% → 50.4%	34.4% → 51.5%
Same baseline class in both series (κ)	76.1% (0.63)	81.7% (0.70)	82.2% (0.70)
Same agent decision in both series (κ)	86.5% (0.75)	87.9% (0.78)	88.8% (0.79)
Tasks whose delivered/withheld status changed	134 (13.5%)	120 (12.1%)	111 (11.2%)

The first Q6_K run was slowed by an environmental fault but its accuracy is unaffected. Agreement statistics were computed before the manual review of the final series.

The three final runs are related in the same way. Table 28 compares each pair of quantization runs task by task: a task's outcome in one run strongly predicts its outcome in another, which is why the

runs are treated as a robustness check rather than as independent replications (Section 7.1). Of the 990 tasks, the baseline hallucinated on 412 in all three runs, on 363 in none, and on 215 in one or two.

Table 28**:** Agreement between the three final runs, task by task (990 tasks; Cohen's κ in brackets).

Per-task outcome	Q4_K_M vs Q6_K	Q4_K_M vs Q8_0	Q6_K vs Q8_0
Baseline hallucinated (yes / no)	84.4% (0.69)	84.9% (0.70)	87.2% (0.74)
Baseline class	69.3% (0.51)	69.6% (0.51)	81.4% (0.68)
Agent delivered or withheld	82.1% (0.62)	83.1% (0.64)	86.3% (0.71)

Computed by `Findings/EvaluateBenchmark/stage_analysis.py` from the per-task evaluation files.

E.11 Secondary outcomes and effect sizes

Table 29**:** Secondary outcomes (exact McNemar tests; intervals over tasks).

Outcome		Q4_K_M	Q6_K	Q8_0
Class A or B reaching the user	rate	59.8% → 11.1%	56.5% → 9.4%	61.0% → 9.5%
	p	1.5×10^{-107}	6.1×10^{-112}	4.7×10^{-128}
Any error (A, B or runtime failure)	rate	74.9% → 11.5%	62.8% → 10.1%	65.4% → 9.8%
	p	1.9×10^{-164}	5.4×10^{-134}	2.9×10^{-140}
Correct code reaching the user (H2)	rate	24.9% → 50.9%	36.9% → 50.4%	34.4% → 51.5%
	gain [CI]	+26.0 [22.7, 29.2]	+13.5 [10.7, 16.5]	+17.1 [13.9, 20.2]
	p	3.2×10^{-51}	6.4×10^{-20}	5.7×10^{-26}
Class B alone reaching the user	count	79 → 94	56 → 70	58 → 74

Hallucinations and runtime failures fall sharply, so the combined error rates fall, but class B errors alone rise slightly: a verifier that checks the API contract does not check the task (Section 8.4). Figure 22 shows the paired differences with both kinds of interval.

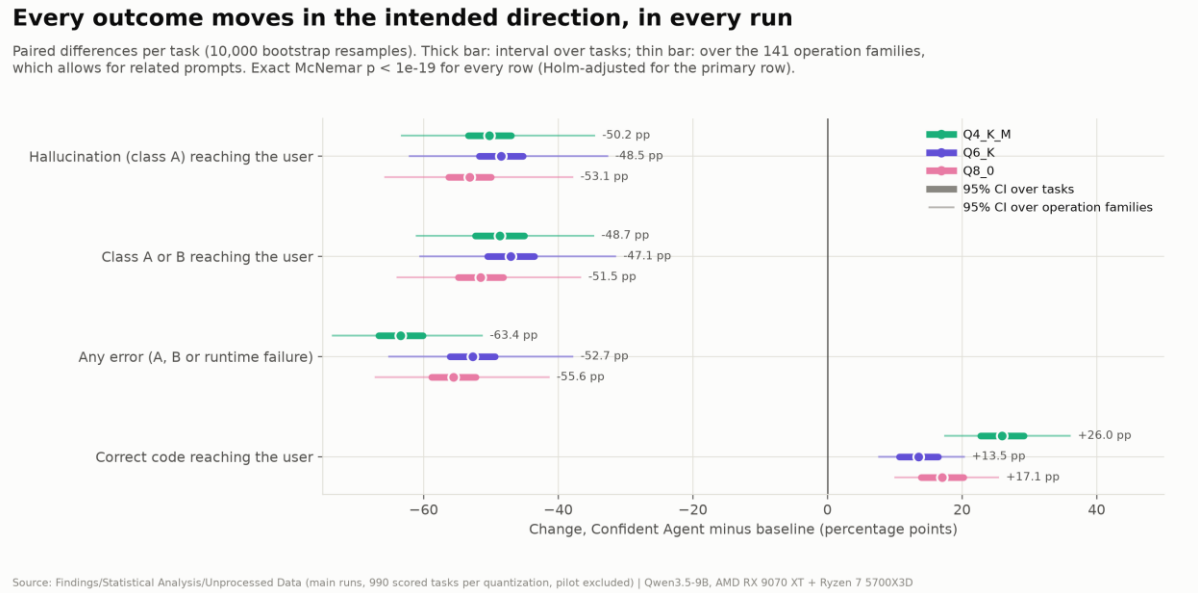


Figure 22**:** Paired differences for the primary and secondary outcomes, with 95% bootstrap intervals over tasks (thick) and operation families (thin).

E.12 Cost statistics and reviewer agreement

H4 is quantified rather than tested for direction. Paired Wilcoxon signed-rank tests on time, energy and tokens give $p < 10^{-160}$ in every one of 18 comparisons with rank-biserial correlations of at least 0.995; the ratio-of-means intervals are narrow (for example, time ×6.36 [6.13, 6.58] at Q4_K_M). Quantization affects cost significantly in both conditions (Kruskal–Wallis, Holm-adjusted pairwise tests significant for time and energy), but, with one run per quantization, those differences cannot be separated from day-to-day machine conditions.

Agreement between the AI-assisted first review and the independent second review on the 46 decisive candidates was 43/46, Cohen's κ = 0.88, which is "almost perfect" agreement on the Landis and Koch (1977) scale. Agreement between repeated runs of the same model and settings was κ = 0.63–0.70 for the baseline class and 0.75–0.79 for the agent's decision ("substantial").

Appendix F: Example Task and Prompts

Task `REST_PY_GH_0003` (operation family `06ffd07d…`, difficulty heuristic "medium", provider specification `github.openapi.json`, SHA-256 `4fbdc7d0…`). The model sees only the instruction:

```
Write a Python function solve(inputs) for GitHub REST API version 2026-03-10. Delete an
organization. The inputs dictionary provides: organization: The organization name. Use the
caller-supplied credential in inputs["credential"], assumed to have the necessary
permissions.
Return the decoded JSON response. Return None for a successful empty response. Raise an
exception
for an unsuccessful HTTP response. Assume referenced resources and accounts exist. Do not
execute
the function.
```

The answer key holds the expected request for the test inputs `{"organization": "example-org", "credential": "EXAMPLE_CREDENTIAL_NOT_REAL"}`:

```
DELETE https://api.github.com/orgs/example-org
Accept: application/vnd.github+json
X-GitHub-Api-Version: 2026-03-10
Authorization: Bearer EXAMPLE_CREDENTIAL_NOT_REAL
```

and a reference solution, generated from the specification by template:

```
import requests
from urllib.parse import quote

def solve(inputs):
    url = 'https://api.github.com/orgs/{org}'
    headers = {}
    headers['Accept'] = 'application/vnd.github+json'
    headers['X-GitHub-Api-Version'] = '2026-03-10'
    headers['Authorization'] = 'Bearer ' + inputs["credential"]
    url = url.replace('{org}', quote(str(inputs['organization']), safe=''))
    response = requests.request('DELETE', url, params={}, headers=headers, timeout=20)
    response.raise_for_status()
    if response.status_code == 204 or not response.content:
        return None
    return response.json()
```

The Confident Agent system prompt (194 lines, in `Code Submission/confident-agent-toolbox/instructions`) prescribes this workflow: write the complete candidate with `requests`; do not look for or write a specification (evidence is selected by host from the eight listed hosts); call `verify_candidate` exactly once with the code only; if a layer is `verified_incorrect` on the first candidate, fix exactly those problems and verify once more; otherwise, or after the second result, reply with the tool's `report` verbatim. It states that the tool does not check task behaviour and that the EWCS is not a probability.

Appendix G: Run Artefacts and Provenance

Table 30**:** Identifiers of the frozen inputs (SHA-256 prefixes; full hashes in the run manifests and the dataset).

Artefact	SHA-256	Notes
Task file (answer key) rest_api_tasks_v1_DRAFT.jsonl	7a09f2bb…	LF line endings
Prompt file rest_api_prompts_v1_DRAFT.jsonl	c4c68a64…	identical in both conditions
Confident Agent system prompt (as sent)	0ebc4dce…	operator note removed
Frozen PyPI evidence (requests)	ef06670a…	interface layer
github.openapi.json	4fbdc7d0…	13.0 MB, 819 paths
petstore.openapi.json	00441c05…	OpenAPI 3.0.4
openmeteo-forecast / air-quality / marine	f57c2918… / ee203da7… / 096a28c7…	converted from YAML
jsonplaceholder / dummyjson / restcountries (curated)	d2dfbc53… / d313813a… / d02f4ab3…	researcher-authored subsets

Run folders (`Findings/Benchmark/Completed Runs/<model>/`) hold one zip per condition with every reply, tool call, verified candidate and verification result, and a run manifest recording the settings, LM Studio's model information, the machine and the power sensors. The final runs were started from repository commits `d954dc0` (Q6_K), `f6ee6b7` (Q8_0) and `7e928f2` (Q4_K_M); the toolbox and benchmark code are identical in all three (benchmark 2026-09-27.2, toolbox 3.0.0).

Appendix H: Error Kinds Used by the Evaluator

Table 31**:** Kinds of error recorded in the code reaching the user, read from the evaluator's findings with fixed rules.

Class	Kinds
A	wrong host, version or base path; invented endpoint or method; invented parameter or body field; invalid value or structure; missing required part or credential; undocumented response field; non-existent `requests` API
B	different declared endpoint; method not acceptable for the task; omitted parameter the task specifies; different parameter value; omitted request body; unrequested request body; request body differs; extra requests; credential missing or malformed where the API's requirement is unknown; JSON body sent as form data; insecure scheme (http)
Runtime failure	input used as the wrong type; reads an input the task does not provide; syntax error; makes no request; does not finish (timeout); blocked operation; undefined name; raises its own error before the request; wrong call or argument types

Appendix I: Manual Review Rules

Each question that a curated specification left open was answered once and applied to every candidate asking it (`Findings/EvaluateBenchmark/reviews/review_rules.py`). The rules, with their evidence:

- **DummyJSON query parameters.** The documentation lists `limit`, `skip`, `select`, `sortBy`, `order` and `q`. Names such as `sort`, `sort_by`, `page`, `fields`, `include` and `_fields` are class A; a live check showed they have no effect (`?sort=title` returns the unsorted list, `?sortBy=title` sorts it).
- **DummyJSON and JSONPlaceholder body fields.** A field the resource has (for example `gender` on a user, `id` on a todo) is an extra documented body field, class B; a field it does not have (for example `title` on a DummyJSON todo, `userId` on a JSONPlaceholder photo) is class A.
- **Response fields.** DummyJSON returns the resource object itself; reading `users`, `post`, `success`, `statusCode` or `content` from a single-resource response is class A.
- **JSONPlaceholder filters and routes.** Filtering by any resource field is documented ("basic filtering is supported through query parameters") and works live; filtering by a non-field (`ids`) is ignored and class A. Nested routes such as `/posts/1/comments` are documented and real; using one where the task expected another endpoint is class B.
- **REST Countries paths.** Every v5 endpoint lies under `/countries/v5` (list, search, aggregate, read and search by property); paths such as `/v5/countries`, `/v5.0/search` or `/v3.1/all` on `api.restcountries.com` do not exist and are class A.

The first review classified the 46 candidates that reached the user as 24 class A and 22 class B; the second reviewer agreed on 43, and the group settled the three JSONPlaceholder list-endpoint cases as correct. Of the 350 withheld candidates, 327 were class A and 23 class B.

Appendix J: Reproducing the Results

The submission folder `COMP301 Project Final Submission` contains `Code Submission` (toolbox, dataset builder), `Findings` (benchmark with completed runs, evaluator, statistical analysis) and `Final Report and Documentation`. The evaluator and statistics need only Python 3.9 or later; the charts need pandas, NumPy, SciPy, Matplotlib and seaborn.

```
cd "COMP301 Project Final Submission\Findings\EvaluateBenchmark"
python evaluate_benchmark.py              # sandboxed scoring of every run (about 12
minutes)
python compile_stats.py                   # CSV tables in Statistical Analysis\Unprocessed
Data
python stage_analysis.py                  # first candidates, agreement between runs
(Section 8.3)
python audit_analysis.py                  # answer-key and evaluator audits (Section 7.6)
cd "..\Statistical Analysis"
python -X utf8 tempstats\build_tempstats.py                # cost charts and tables
python -X utf8 tempstats\report_figures\build_report_figures.py   # charts R01-R05
cd "..\..\Final Report and Documentation\Final Report\build"
python -X utf8 build_diagrams.py          # diagrams D01-D08
node build_report.js report.docx          # this report (then update fields in Word)
```

Rerunning the benchmark itself requires LM Studio with the Qwen3.5-9B quantizations, the toolbox's Python environment and LibreHardwareMonitor for energy; the procedure is in `Findings/Benchmark/README.md`.

Appendix K: Glossary

Table 32**:** Glossary.

Term	Meaning
Abstention	The agent withholds code because evidence contradicts it or cannot establish it.
Baseline	The same model with a minimal system prompt and no verification.
Class A / B / C	API hallucination / valid call that is wrong for the task / equivalent to the expected request (Table 1).
Confident Agent	The model plus the Confident Agent prompt and the `verify_candidate` tool.
Decision policy	The ordered rules that turn verification results into deliver, deliver with flag, correct or abstain: safety constraint, uncertainty gate, then thresholds.
EWCS	Evidence-Weighted Confidence Score: the share of applicable verification weight supported by positive evidence. Not a probability.
First candidate	The first code the model submitted for verification in a task, written under the Confident Agent prompt before any verification feedback.
Frozen evidence	Specification files stored with their hash and never changed during the experiment.
Incorrect acceptance / rejection	A hallucinated candidate delivered / a correct candidate withheld.
McNemar test	Test for a change in a paired binary outcome, using only the discordant pairs.
Operation family	A group of tasks that exercise the same underlying API operation.
Safety constraint	The policy rule that never releases code with a layer verified incorrect.
Uncertainty gate	The policy rule that withholds code when any applicable verification layer is unverifiable.
Verified correct / incorrect / unverifiable / not applicable	The four states of a verification layer.